

INSTITUTO FEDERAL
Fluminense

Campus
Bom Jesus do Itabapoana

Algoritmos e estruturas de dados II

Árvores

Professora Ana Mara de Oliveira Figueiredo
ana.figueiredo@iff.edu.br

Definição

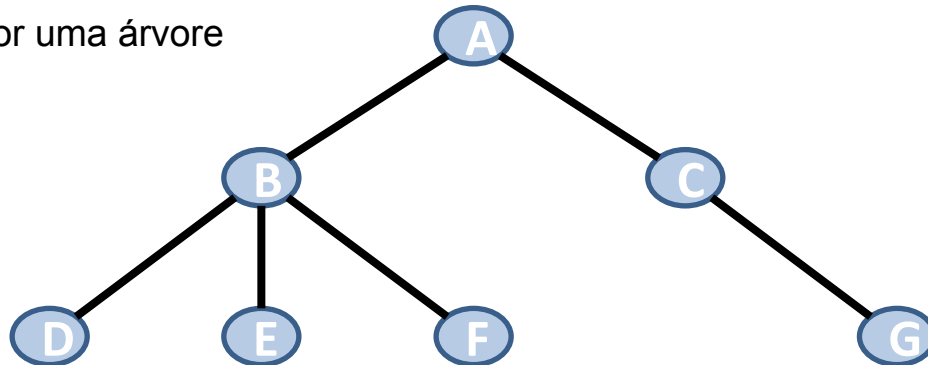
- Diversas aplicações necessitam que seja representado um conjunto de objetos e as suas relações hierárquicas
- Uma árvore é uma abstração matemática usada para representar estruturas hierárquicas não lineares dos objetos modelados

Definição

- Árvores: É um tipo especial de grafo
 - Definida usando um conjunto de nós (ou vértices) e arestas
 - Qualquer par de vértices está conectado a apenas uma aresta
 - Grafo não direcionado, conexo e acíclico (sem ciclos)

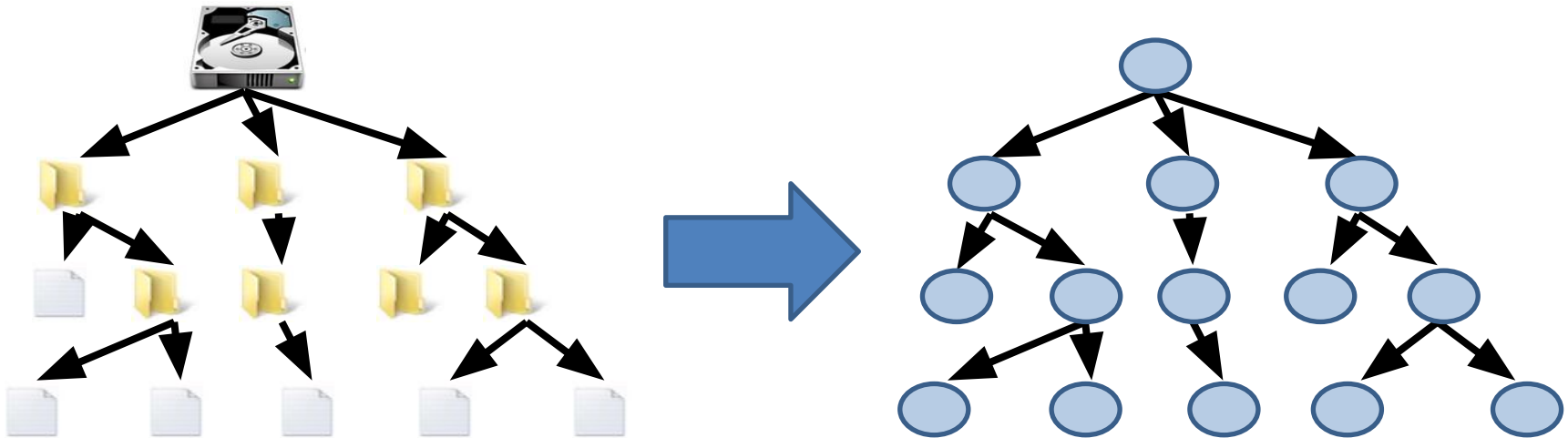
- Vértice

- Cada uma das entidades representadas na árvore (depende da natureza do problema).
- Basicamente, qualquer problema em que exista algum tipo de hierarquia pode ser representado por uma árvore



Definição

- Exemplo
 - Estrutura de pastas do computador



Definição

- Exemplos

- relações de descendência (pai, filho, etc.)
- diagrama hierárquico de uma organização;
- campeonatos de modalidades desportivas;
- taxonomia

- Em computação

- busca de dados armazenados no computador
- representação de espaço de soluções
 - Exemplo: jogo de xadrez;
- modelagem de algoritmos

Conceitos básicos

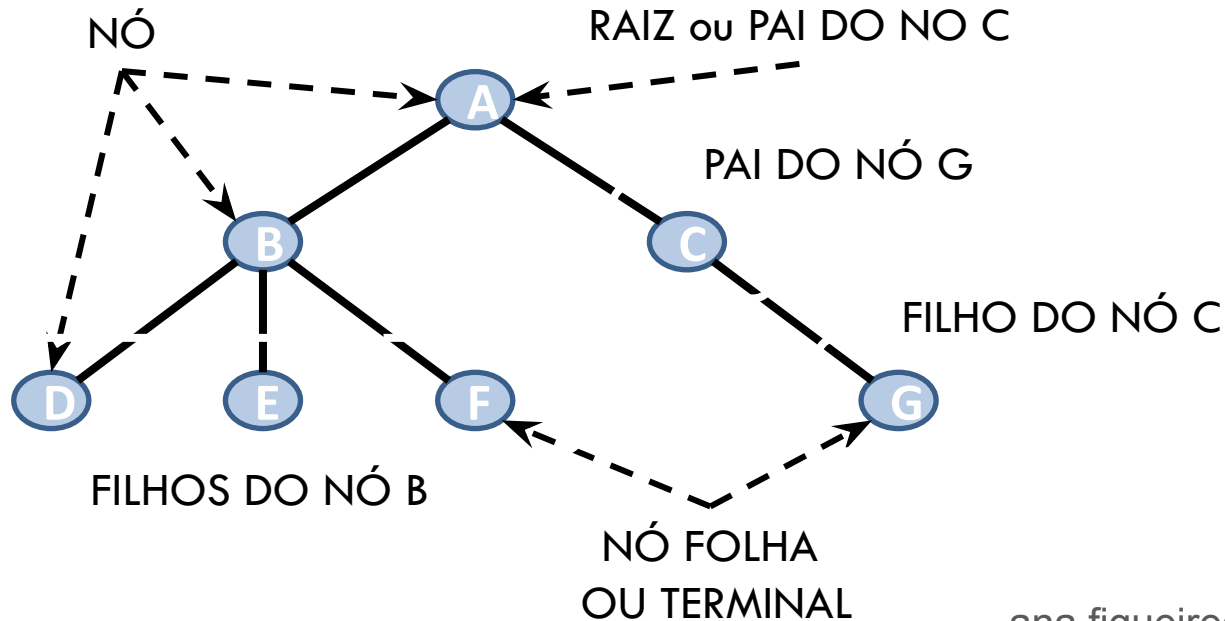
- Principais conceitos relativos às árvores
 - Raiz
 - nó mais alto na árvore, o único que não possui pai
 - Pai ou ancestral
 - nó antecessor imediato de outro nó
 - Filho
 - é o nó sucessor imediato de outro nó

Conceitos básicos

- Principais conceitos relativos às árvores
 - Nó folha ou terminal
 - qualquer nó que não possui filhos
 - Nó interno ou não-terminal
 - nó que possui ao menos UM filho
 - Caminho
 - sequência de nós de modo que existe sempre uma aresta ligando o nó anterior com o seguinte

Conceitos básicos

- Exemplo



Conceitos básicos

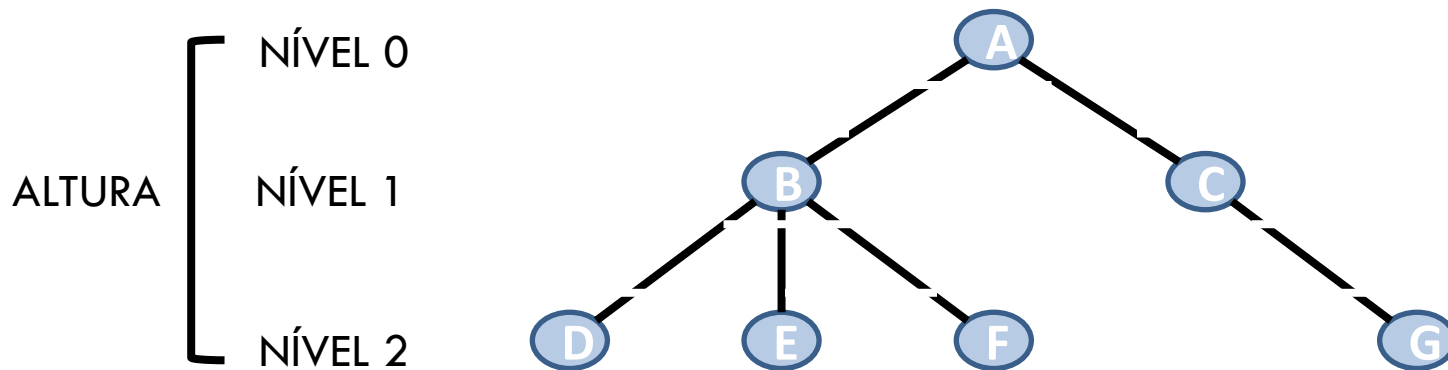
- Observação
 - Dado um determinado nó da árvore, cada filho seu é considerado a **raiz** de uma nova **sub-árvore**
 - Qualquer nó é a **raiz** de uma **sub-árvore** consistindo dele e dos nós abaixo dele
 - Conceito recursivo

Conceitos básicos

- Principais conceitos relativos às árvores
 - Nível
 - É dado pelo o número de nós que existem no caminho entre esse nó e a raiz (nível 0)
 - Nós são classificados em diferentes níveis
 - Altura
 - Também chamada de profundidade
 - Número total de níveis de uma árvore
 - Comprimento do caminho mais longo da raiz até uma das suas folhas

Conceitos básicos

- Nível e altura

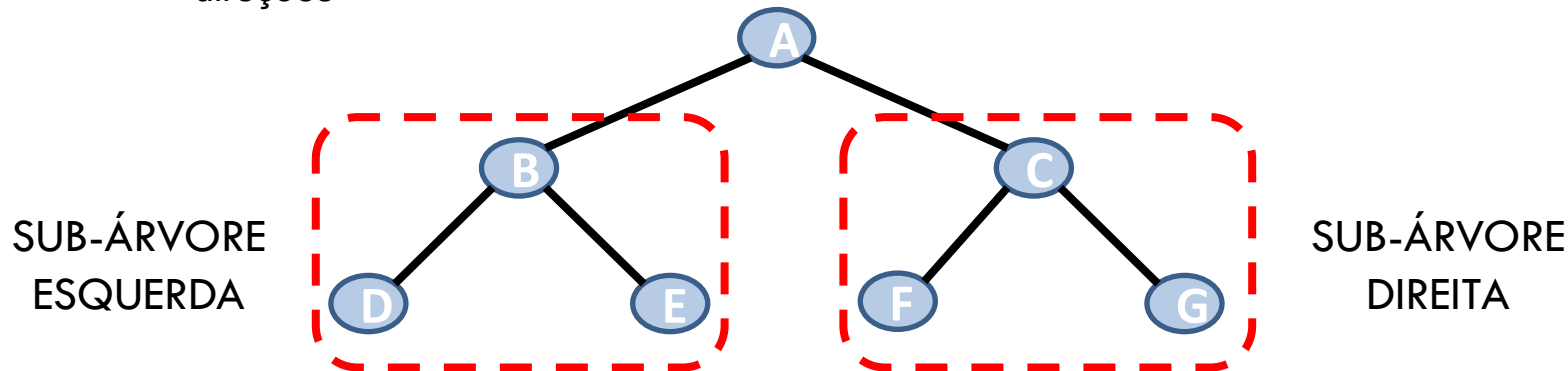


Tipos de árvores

- Na computação, assim como na natureza, existem vários tipos diferentes de árvores.
 - Cada uma delas foi desenvolvida pensando diferentes tipos de aplicações
 - árvore binária de busca
 - árvore AVL
 - árvore Rubro-Negra
 - árvore B, B+ e B*
 - árvore 2-3
 - árvore 2-3-4
 - quadtree
 - octree

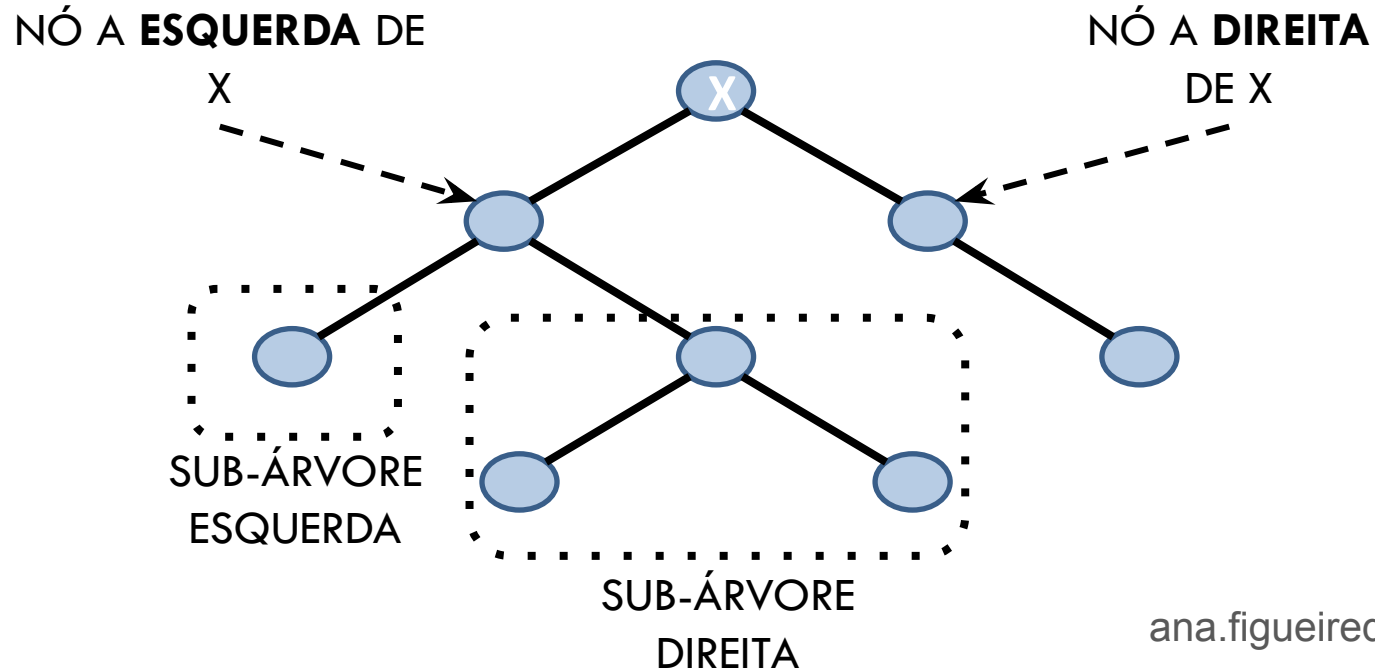
Árvore Binária

- É um tipo especial de árvore
 - Cada nó pode possuir nenhuma, uma ou no máximo duas **sub-árvores**
 - Sub-árvore da **esquerda** e a da **direita**
 - Usadas em situações onde, a cada passo, é preciso tomar uma decisão entre duas direções



Árvore Binária

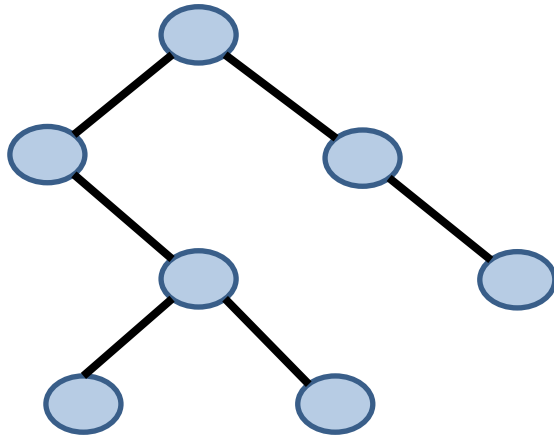
- Exemplo de árvore binária



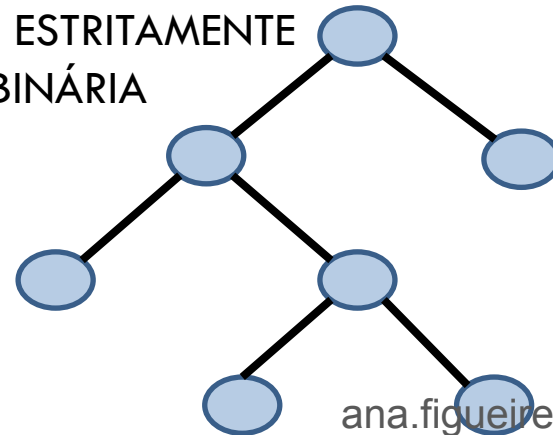
Árvore Binária

- Árvore estritamente binária
 - Cada nó possui sempre ou 0 (no caso de nó folha) ou 2 sub-árvores
 - Nenhum nó tem **filho único**

ÁRVORE
BINÁRIA

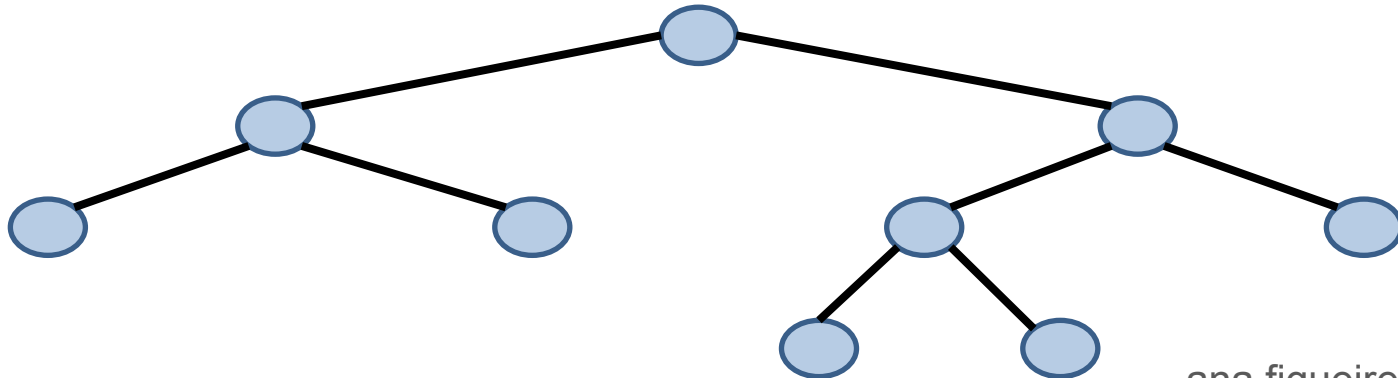


ÁRVORE ESTRITAMENTE
BINÁRIA



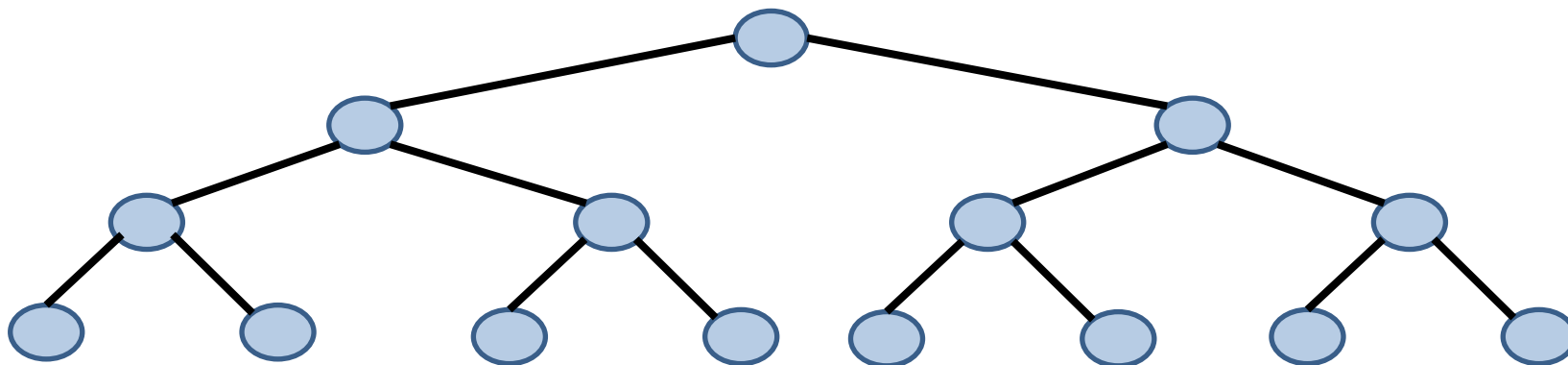
Árvore Binária

- Árvore binária completa
 - A diferença de altura entre as sub-árvores de qualquer nó é no máximo 1
 - Se a altura da árvore é **D**, cada nó folha está no nível **D** ou **D-1**



Árvore Binária

- Árvore binária cheia
 - Árvore estritamente binária onde todos os nó folhas estão no mesmo nível



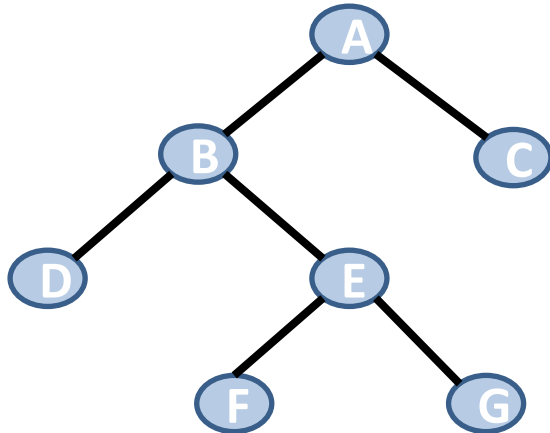
Nível (n)	0	1	2	3
Nº de nós	1	2	4	8
Potência	2^0	2^1	2^2	2^3

Tipo de representação

- Como implementar uma árvore no computador?
- Existem duas abordagens muito utilizadas
 - Usando um array (alocação estática)
 - Usando uma lista encadeada (alocação dinâmica)

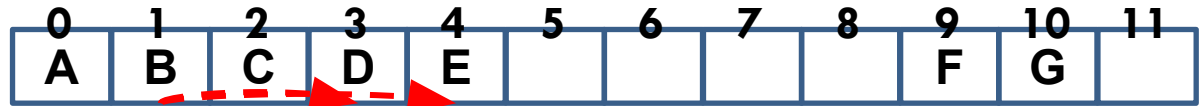
Tipo de representação

- Usando um array (alocação estática)
 - Necessário definir o número máximo de nós
 - Tamanho do array
 - Usa 2 funções para retornar a posição dos filhos à esquerda e à direita de um pai



$$\text{FILHO_ESQ}(\text{PAI}) = 2 * \text{PAI} + 1$$

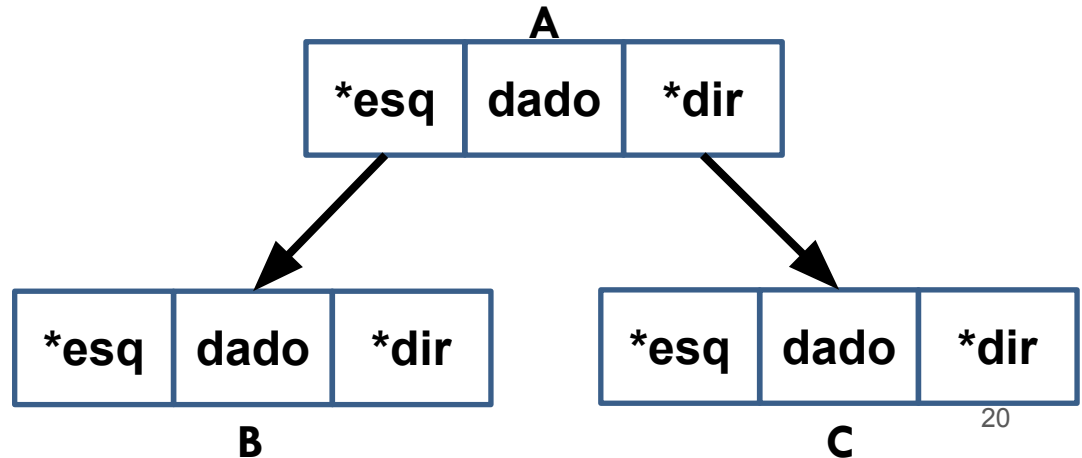
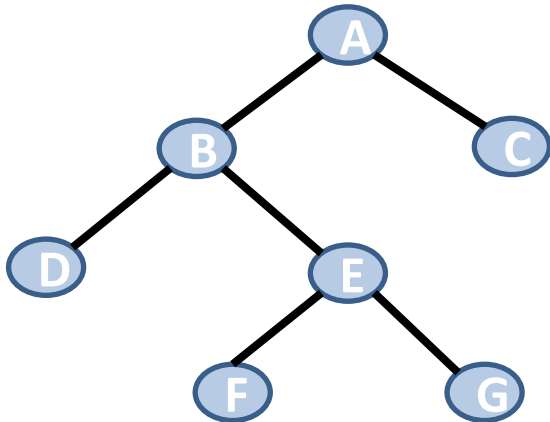
$$\text{FILHO_DIR}(\text{PAI}) = 2 * \text{PAI} + 2$$



FILHOS

Tipo de representação

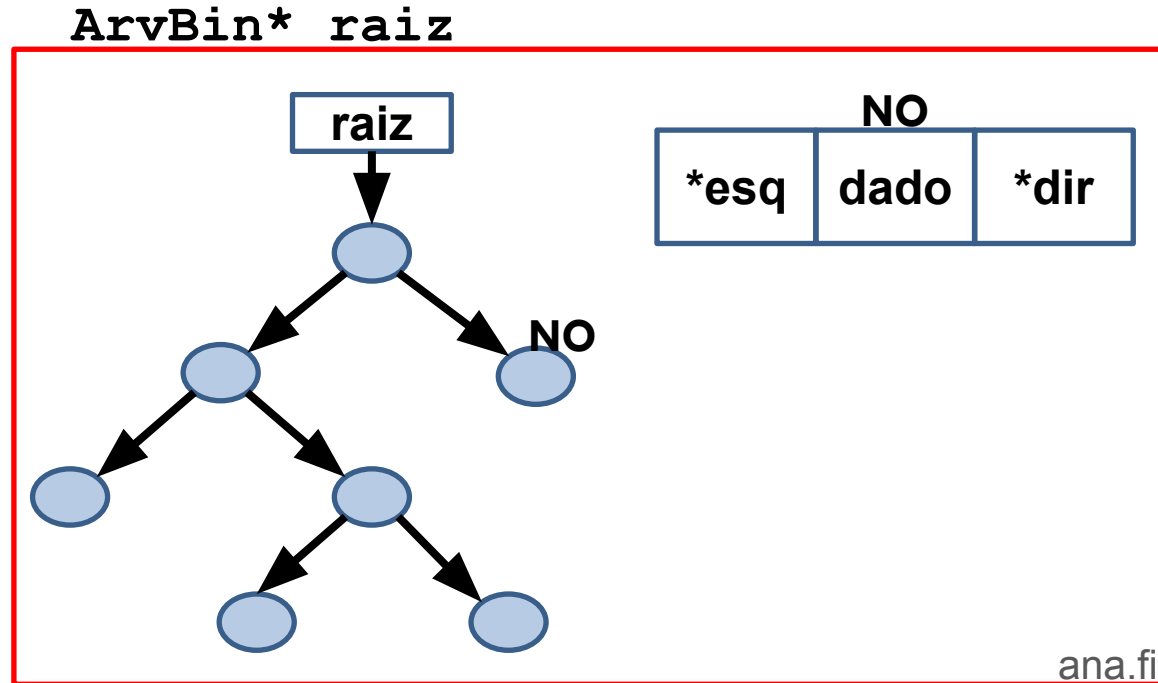
- Lista encadeada (alocação dinâmica)
 - Espaço de memória alocado em tempo de execução
 - A árvore cresce à medida que novos elementos são armazenados, e diminui à medida que elementos são removidos



TAD Árvore Binária

- Definição
 - Uso de alocação dinâmica
 - Para guardar o primeiro nó da árvore utilizamos um **ponteiro para ponteiro**
 - Um **ponteiro para ponteiro** pode guardar o endereço de um **ponteiro**
 - Assim, fica fácil mudar quem é a **raiz** da árvore (se necessário)

- Definição



TAD Árvore Binária

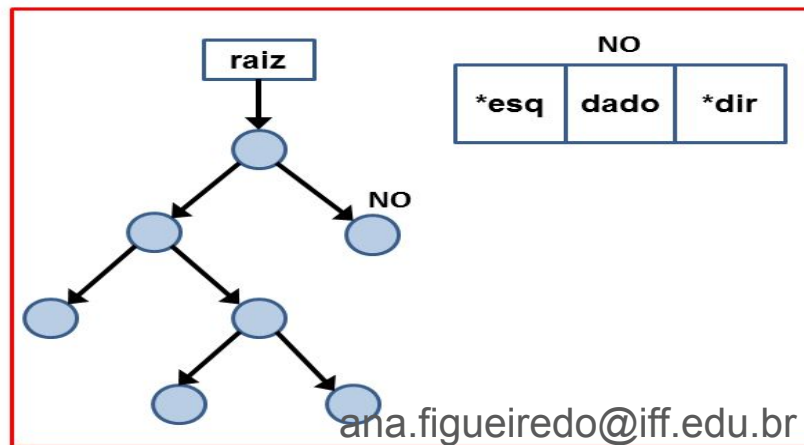
- Definição

```
//Arquivo ArvoreBinaria.h
typedef struct NO* ArvBin;

//Arquivo ArvoreBinaria.c
#include <stdio.h>
#include <stdlib.h>
#include "ArvoreBinaria.h" //inclui os Protótipos
struct NO{
    int info;
    struct NO *esq;
    struct NO *dir;
};

//programa principal
ArvBin* raiz; //ponteiro para ponteiro
```

ArvBin* raiz

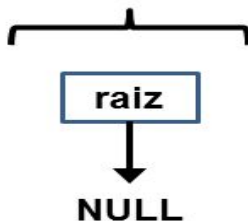


TAD Árvore Binária

- Criando a árvore

```
1 //arquivo ArvoreBinaria.h
2 ArvBin* cria_ArvBin();
3
4 //arquivo ArvoreBinaria.c
5 ArvBin* cria_ArvBin(){
6     ArvBin* raiz = (ArvBin*) malloc(sizeof(ArvBin));
7     if(raiz != NULL)
8         *raiz = NULL;
9     return raiz;
10 }
11
12 //programa principal
13 ArvBin* raiz = cria_ArvBin();
```

ArvBin* raiz



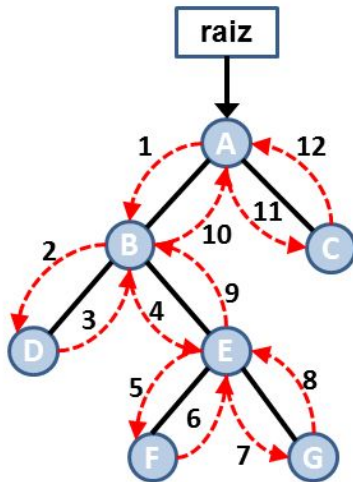
TAD Árvore Binária

- Liberando a árvore
 - Uso de 2 funções: uma percorre e libera os nós, outra trata a raiz

```
5 void libera_NO(struct NO* no) {  
6     if(no == NULL)  
7         return;  
8     libera_NO(no->esq);  
9     libera_NO(no->dir);  
10    free(no);  
11    no = NULL;  
12 }  
13 void libera_ArvBin(ArvBin* raiz) {  
14     if(raiz == NULL)  
15         return;  
16     libera_NO(*raiz); //libera cada nó  
17     free(raiz); //libera a raiz  
18 }
```

TAD Árvore Binária

- Remoção: passo a passo



	1	visita B
	2	visita D
✗	3	libera D, volta para B
	4	visita E
	5	visita F
✗	6	libera F, volta para E
	7	visita G
✗	8	libera G, volta para E
✗	9	libera E, volta para B
✗	10	libera B, volta para A
	11	visita C
✗	12	libera C, volta para A e
✗		libera A
✗		libera raiz

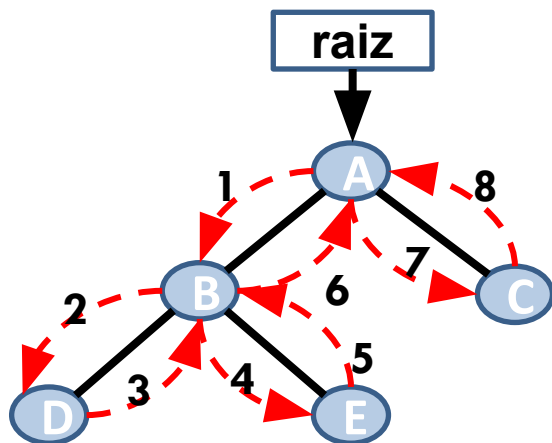
TAD Árvore Binária

- Informações básicas sobre a árvore
 - Altura
 - Número total de níveis de uma árvore

```
5  int altura_ArvBin(ArvBin *raiz) {  
6      if (raiz == NULL)  
7          return 0;  
8      if (*raiz == NULL)  
9          return 0;  
10     int alt_esq = altura_ArvBin(&((*raiz)->esq));  
11     int alt_dir = altura_ArvBin(&((*raiz)->dir));  
12     if (alt_esq > alt_dir)  
13         return (alt_esq + 1);  
14     else  
15         return (alt_dir + 1);  
16 }
```

TAD Árvore Binária

- Informações básicas sobre a árvore
 - Altura



	inicia no nó A
1	visita B
2	visita D
3	D é nó folha: altura é 1. Volta para B
4	visita E
5	E é nó folha: altura é 1. Volta para B
6	altura de B é 2: maior altura dos filhos + 1. Volta para A
7	visita C
8	C é nó folha: altura é 1. Volta para A
	altura de A é 3: maior altura dos filhos + 1.

TAD Árvore Binária

- Informações básicas sobre a árvore

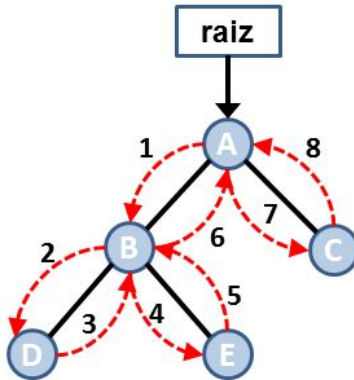
- Número de nós

- Quantidade de elementos na árvore

```
5  int totalNO_ArvBin(ArvBin *raiz) {  
6      if (raiz == NULL)  
7          return 0;  
8      if (*raiz == NULL)  
9          return 0;  
10     int alt_esq = totalNO_ArvBin(&((*raiz)->esq));  
11     int alt_dir = totalNO_ArvBin(&((*raiz)->dir));  
12     return(alt_esq + alt_dir + 1);  
13 }
```

TAD Árvore Binária

- Informações básicas sobre a árvore
 - Número de nós



1	visita B
2	visita D
3	D é nó folha: conta como 1 nó. Volta para B
4	visita E
5	E é nó folha: conta como 1 nó. Volta para B
6	Número de nós em B é 3: total de nós a direita (1) + total de nós a esquerda (1) + 1. Volta para A
7	visita C
8	C é nó folha: conta como 1 nó. Volta para A
Número de nós em A é 5: total de nós a direita (1) + total de nós a esquerda (3) + 1.	

Percurso na árvore

- Percorrer todos os nós é uma operação muito comum em árvores binárias
 - Cada nó é visitado uma única vez
 - Isso gera uma sequência linear de nós, cuja ordem depende de como a árvore foi percorrida
 - Não existe uma **ordem natural** para se percorrer todos os nós de uma árvore binária
 - Isso pode ser feito para executar alguma ação em cada nó
 - Essa ação pode ser mostrar (imprimir) o valor do nó, modificar esse valor, etc.

Percurso na árvore

- Podemos percorrer a árvore de 3 formas
 - Percurso pré-ordem
 - visita a **raiz**, o filho da **esquerda** e o filho da **direita**
 - Percurso um-ordem
 - visita o filho da **esquerda**, a **raiz** e o filho da **direita**
 - Percurso pós-ordem
 - visita o filho da **esquerda**, o filho da **direita** e a **raiz**
- Essas são os percursos mais importantes
 - Existem outras formas de percurso

Percurso pré-ordem

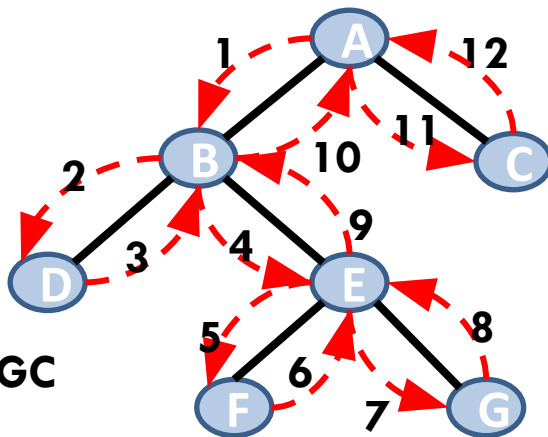
```

5 void preOrdem_ArvBin(ArvBin *raiz)
6 {
7     if(raiz == NULL)
8         return;
9     if(*raiz != NULL){
10        printf("%d\n", (*raiz)->info);
11        preOrdem_ArvBin(&((*raiz)->esq));
12        preOrdem_ArvBin(&((*raiz)->dir));
13    }
14 }

```

● Ordem de visitação

- Raiz
- Filho esquerdo
- Filho direito



RESULTADO: ABDEFGC

	inicia no nó A
1	imprime A, visita B
2	imprime B, visita D
3	imprime D, volta para B
4	visita E
5	imprime E, visita F
6	imprime F, volta para E
7	visita G
8	imprime G, volta para E
9	volta para B
10	volta para A
11	visita C
12	imprime C, volta para A

Percurso em-ordem

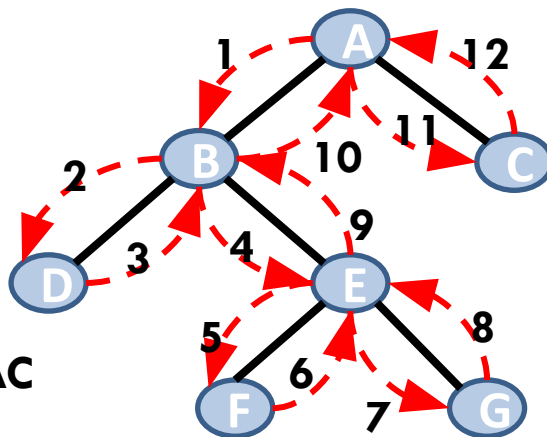
```

5 void emOrdem_ArvBin(ArvBin *raiz)
6 {
7     if(raiz == NULL)
8         return;
9     if(*raiz != NULL){
10        emOrdem_ArvBin(&((*raiz)->esq));
11        printf("%d\n", (*raiz)->info);
12        emOrdem_ArvBin(&((*raiz)->dir));
13    }
14 }

```

• Ordem de visitação

- Filho esquerdo
- Raiz
- Filho direito



RESULTADO: DBFEGAC

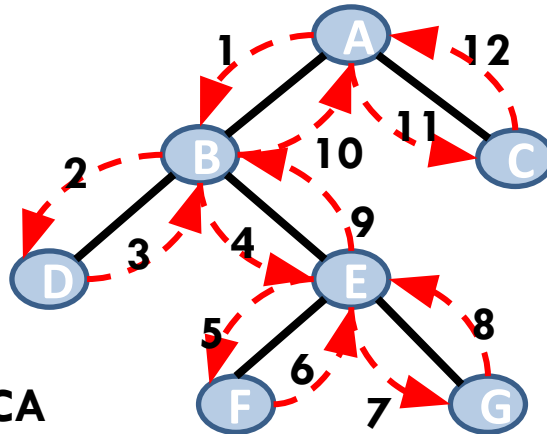
	inicia no nó A
1	visita B
2	visita D
3	imprime D, volta para B
4	imprime B, visita E
5	visita F
6	imprime F, volta para E
7	imprime E, visita G
8	imprime G, volta para E
9	volta para B
10	volta para A
11	imprime A, visita C
12	imprime C, volta para A

Percurso pós-ordem

```
5 void posOrdem_ArvBin(ArvBin *raiz) {
6     if (raiz == NULL)
7         return;
8     if (*raiz != NULL) {
9         posOrdem_ArvBin(raiz->esq);
10        posOrdem_ArvBin(&((*raiz)->dir));
11        printf("%d\n", (*raiz)->info);
12    }
13 }
```

- Ordem de visitação

- Filho esquerdo
- Filho direito
- Raiz



RESULTADO: DFGEBCA

	INICIA NO NÓ A
1	visita B
2	visita D
3	imprime D, volta para B
4	visita E
5	visita F
6	imprime F, volta para E
7	visita G
8	imprime G, volta para E
9	imprime E, volta para B
10	imprime B, volta para A
11	visita C
12	imprime C, volta para A e imprime A

Árvore Binária de Busca

- Definição

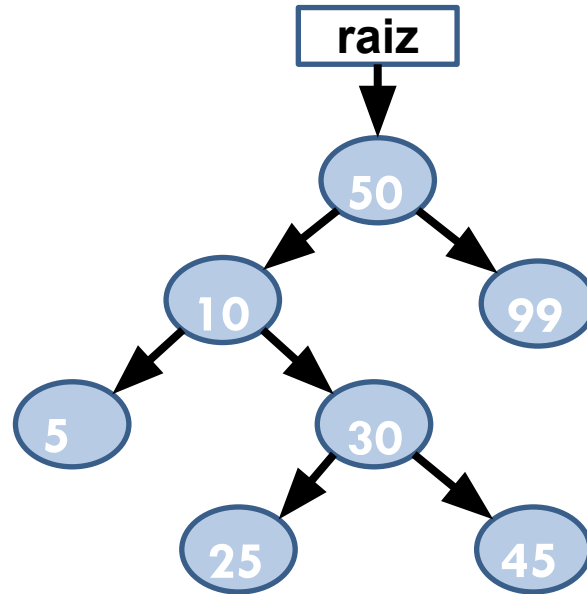
- É uma árvore binária
 - Cada nó pode ter 0, 1 ou 2 filhos
- Cada nó possui da árvore possui um valor (chave) associado a ele
 - Não existem valores repetidos
- Esse valor determina a posição do nó na árvore

Árvore Binária de Busca

- Regra para posicionamento dos valores na árvore
 - Para cada nó pai
 - todos os valores da sub-árvore **esquerda são menores** do que o nó pai
 - todos os valores da sub-árvore **direita são maiores** do que o nó pai;
 - Inserção e remoção devem ser realizadas respeitando essa regra de posicionamento dos nós.

Árvore Binária de Busca

- Exemplo



Árvore Binária de Busca

- Ótima alternativa para operações de busca binária
 - Possui a vantagem de ser uma estrutura dinâmica em comparação ao array
 - É mais fácil inserir valores na árvore do que em um array ordenado
 - Array: envolve deslocamento de elementos

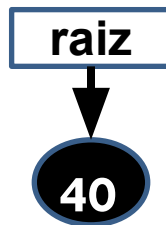
Árvore Binária de Busca - Inserção

- Para inserir um valor **V** na árvore
 - Se a raiz é igual a **NULL**, insira o nó
 - Se **V** é menor do que a raiz: vá para a **sub-árvore esquerda**
 - Se **V** é maior do que a raiz: vá para a **sub-árvore direita**
 - Aplique o método **recursivamente**
 - pode ser feito sem recursão
- Dessa forma, percorremos um conjunto de nós da árvore até chegar ao nó folha que irá se tornar o pai do novo nó

Árvore Binária de Busca: Inserção

- Devemos também considerar a inserção em uma árvore que está vazia

**Insere o valor '40'
em uma árvore
vazia**



```
*raiz = novo;
```

TAD Árvore Binária

- Inserção em uma Árvore Binária de Busca

```
int insere_ArvBin(ArvBin* raiz, int valor){
    if(raiz == NULL)
        return 0;
    struct NO* novo;
    novo = (struct NO*) malloc(sizeof(struct NO));
    if(novo == NULL)
        return 0;
    novo->infor = valor;
    novo->dir = NULL;
    novo->esq = NULL;
    //procurar onde inserir!
```

TAD Árvore Binária

- Inserção em uma Árvore Binária de Busca

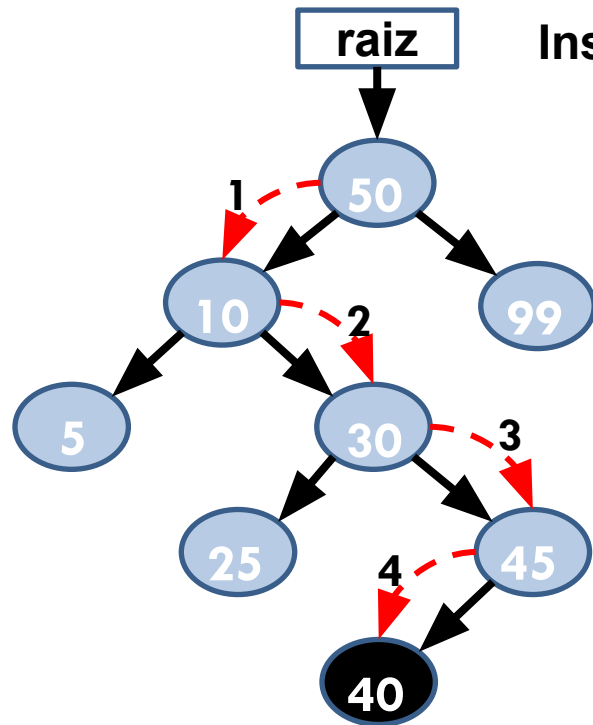
Navega nos nós
da árvore até
chegar em um nó
folha

Insere como filho
desse nó folha

```
if(*raiz == NULL)
    *raiz = novo;
else{
    struct NO* atual = *raiz;
    struct NO* ant = NULL;
    while(atual != NULL){
        ant = atual;
        if(valor == atual->info){
            free(novo);
            return 0; //elemento já existe
        }
        if(valor > atual->info)
            atual = atual->dir;
        else
            atual = atual->esq;
    }
    if(valor > ant->info)
        ant->dir = novo;
    else
        ant->esq = novo;
}
return 1;
```

Árvore Binária de Busca: Inserção

- Exemplo



Insere o valor '40' como um nó folha

1	valor é menor do que 50: visita filho da esquerda
2	valor é maior do que 10: visita filho da direita
3	valor é maior do que 30: visita filho da direita
4	valor é menor do que 45: visita filho da esquerda
Fim	Não existe filho da esquerda. Valor passa a ser o filho da esquerda de 45

Árvore Binária de Busca: Busca

- Consultar se um determinado nó **V** existe em uma árvore é similar a operação de inserção
 - primeiro compare o valor buscado com a **raiz**;
 - se **V** é menor do que a raiz: vá para a **sub-árvore da esquerda**;
 - se **V** é maior do que a raiz: vá para a **sub-árvore da direita**;
 - aplique o método recursivamente até que a raiz seja igual ao valor buscado
 - pode ser feito sem recursão

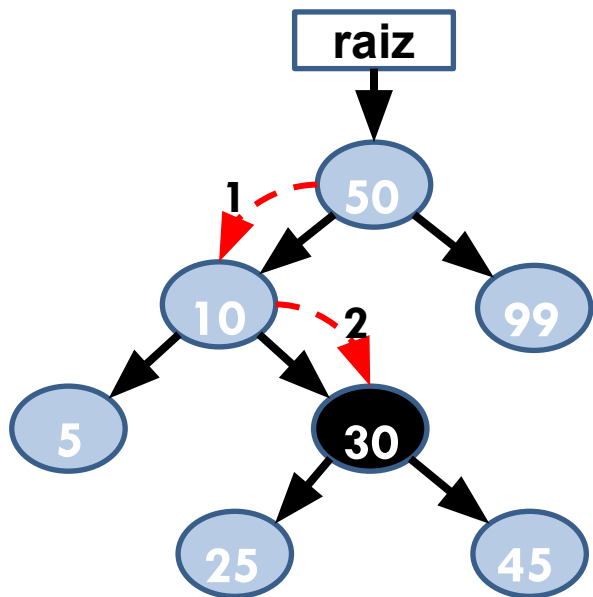
TAD Árvore Binária

- Busca em uma Árvore Binária de Busca

```
6  int consulta_ArvBin(ArvBin *raiz, int valor) {
7      if(raiz == NULL)
8          return 0;
9      struct NO* atual = *raiz;
10     while(atual != NULL) {
11         if(valor == atual->info) {
12             return 1;
13         }
14         if(valor > atual->info)
15             atual = atual->dir;
16         else
17             atual = atual->esq;
18     }
19     return 0;
20 }
```

Árvore Binária de Busca: Busca

- Exemplo

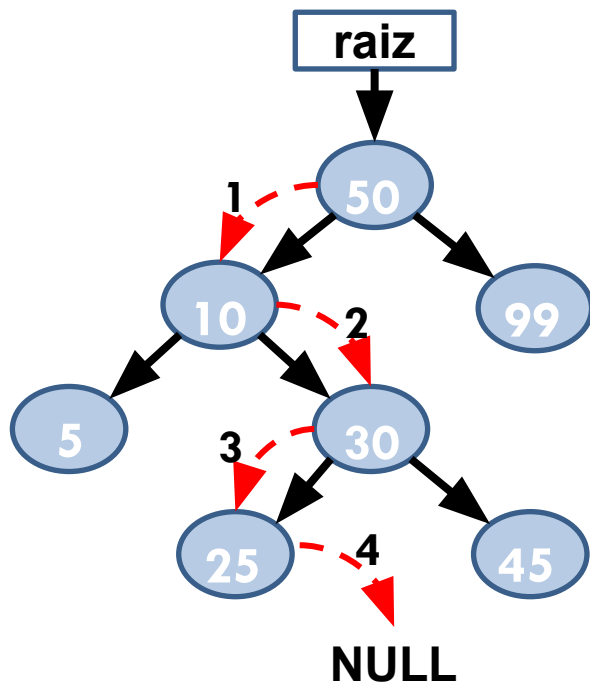


Valor procurado: 30

1	valor procurado é menor do que 50: visita filho da esquerda
2	valor procurado é maior do que 10: visita filho da direita
Fim	valor procurado é igual ao do nó: retornar dados do nó

Árvore Binária de Busca: Busca

- Exemplo: valor buscado não existe!



Valor procurado: 28

1	valor procurado é menor do que 50: visita filho da esquerda
2	valor procurado é maior do que 10: visita filho da direita
3	valor procurado é menor do que 30: visita filho da esquerda
4	valor procurado é maior do que 25: visita filho da direita
Fim	Filho da direita de 25 não existe: a busca falhou

Árvore Binária de Busca: Remoção

- Remover um nó de uma árvore binária de busca não é uma tarefa tão simples quanto a inserção.
 - Isso ocorre porque precisamos procurar o nó a ser removido da árvore o qual pode ser um
 - nó folha
 - nó interno (que pode ser a raiz), com um ou dois filhos.
 - Se for um nó interno
 - Reorganizar a árvore para que ela continue sendo uma árvore binária de busca

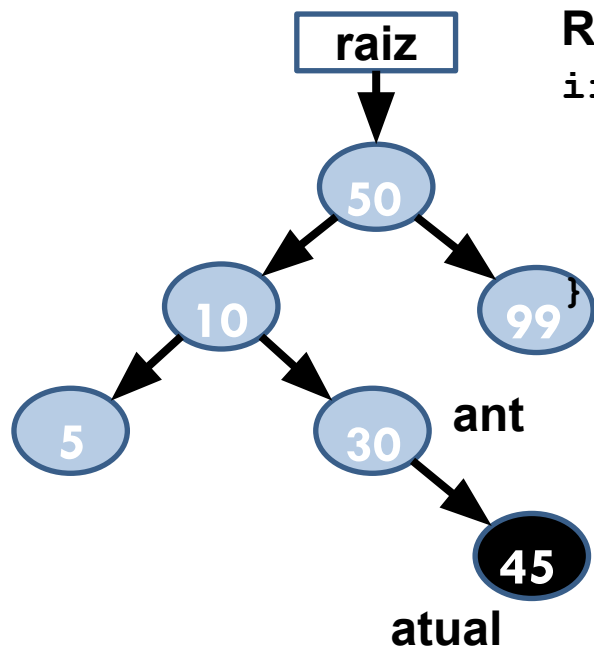
TAD Árvore Binária

- Remoção em uma Árvore Binária de Busca
 - Trabalha com 2 funções
 - Busca pelo nó
 - Tratar os 3 tipos de remoção: com 0, 1 ou 2 filhos

```
8  int remove_ArvBin(ArvBin *raiz, int valor){
9      /*
10     FUNÇÃO RESPONSÁVEL PELA BUSCA
11     DO NÓ A SER REMOVIDO
12     */
13 }
14 struct NO* remove_atual(struct NO* atual){
15     /*
16     FUNÇÃO RESPONSÁVEL POR TRATAR OS 3
17     TIPOS DE REMOÇÃO
18     */
19 }
```

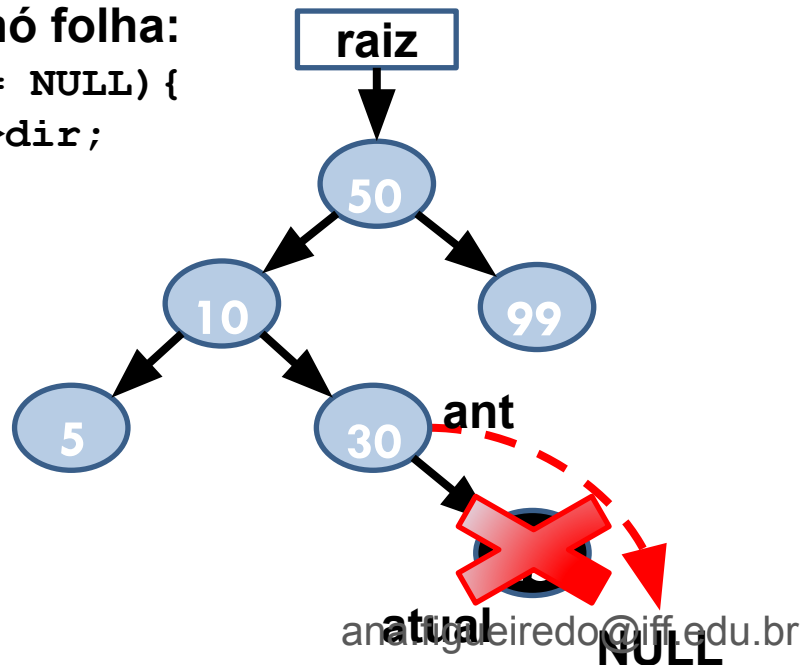
Árvore Binária de Busca: Remoção

- Exemplo: remoção de um nó folha



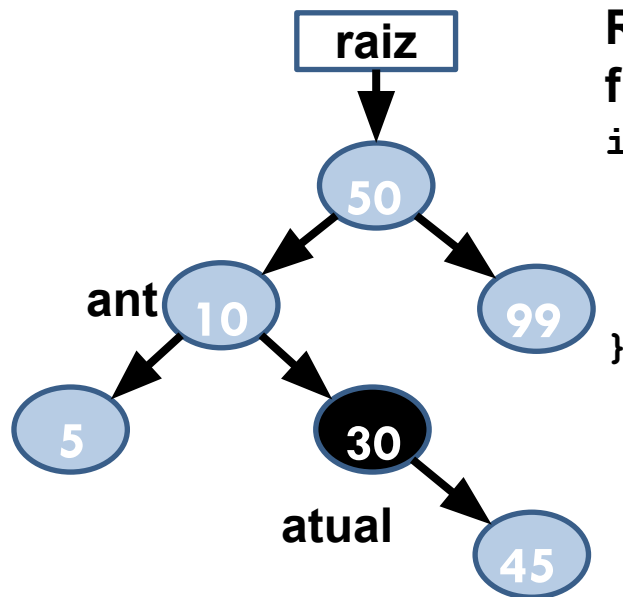
Remoção de um nó folha:

```
if(atual->esq == NULL){  
    no2 = atual->dir;  
    free(atual);  
    return no2;  
}
```



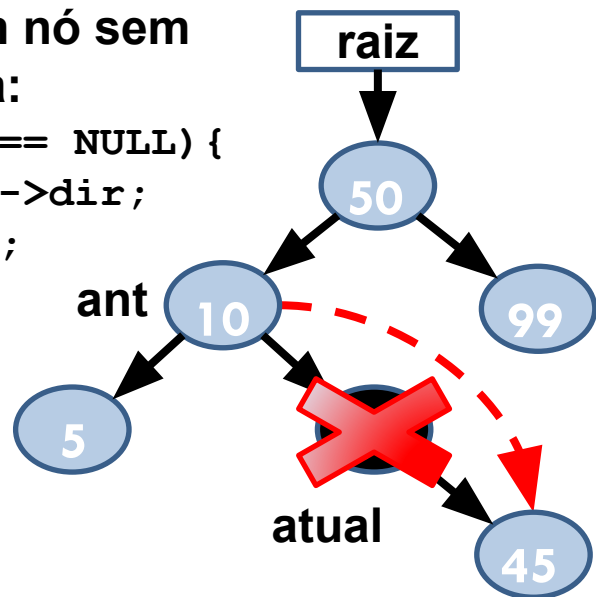
Árvore Binária de Busca: Remoção

- Exemplo: remoção de um nó com 1 filho



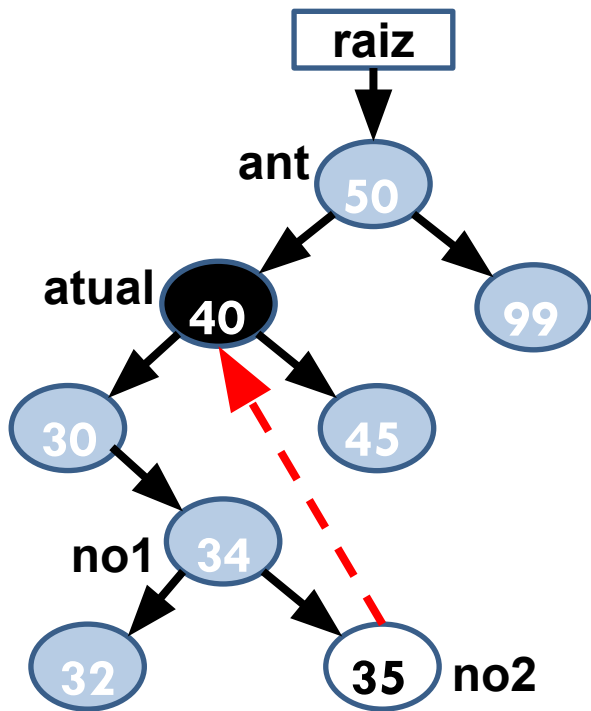
**Remoção de um nó sem
filho a esquerda:**

```
if(atual->esq == NULL) {  
    no2 = atual->dir;  
    free(atual);  
    return no2;  
}
```



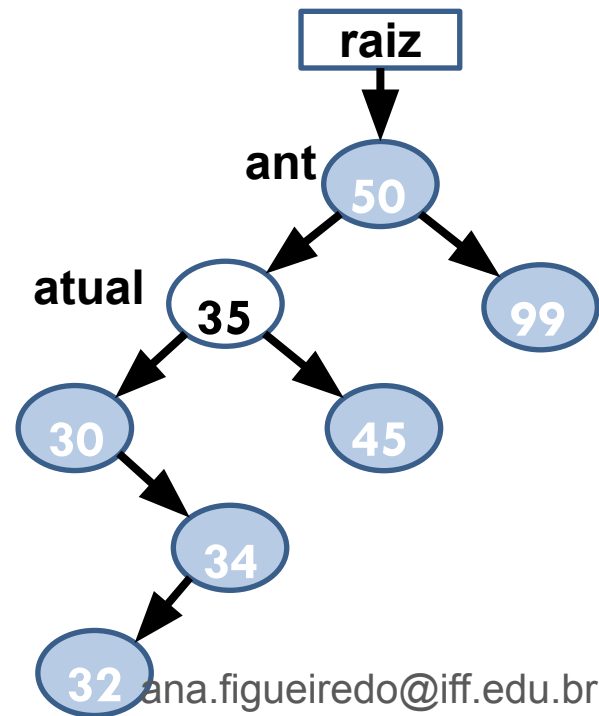
Árvore Binária de Busca: Remoção

- Exemplo: remoção de um nó com 2 filhos



**Remoção de um nó
com ambos os
filhos:**

**Remover o nó
“atual” e substituí-lo
pelo nó “mais a
direita” da sub-
árvore da esquerda
do nó “atual”**



TAD Árvore Binária

- Remoção em uma Árvore Binária de Busca

```
1 struct NO* remove_atual(struct NO* atual) {  
2     struct NO *no1, *no2;  
3     if(atual->esq == NULL) {  
4         no2 = atual->dir;  
5         free(atual);  
6         return no2;  
7     }  
8     no1 = atual;  
9     no2 = atual->esq;  
10    while(no2->dir != NULL) {  
11        no1 = no2;  
12        no2 = no2->dir;  
13    }  
14  
15    if(no1 != atual) {  
16        no1->dir = no2->esq;  
17        no2->esq = atual->esq;  
18    }  
19    no2->dir = atual->dir;  
20    free(atual);  
21    return no2;  
22 }
```

Sem filho da esquerda.

Apontar para o filho da direita (trata nó folha e nó com 1 filho)

Procura filho mais a direita na sub-árvore da esquerda.

Copia o filho mais a direita na sub-árvore da esquerda para o lugar do nó removido.

TAD Árvore Binária

- Remoção em uma Árvore Binária de Busca

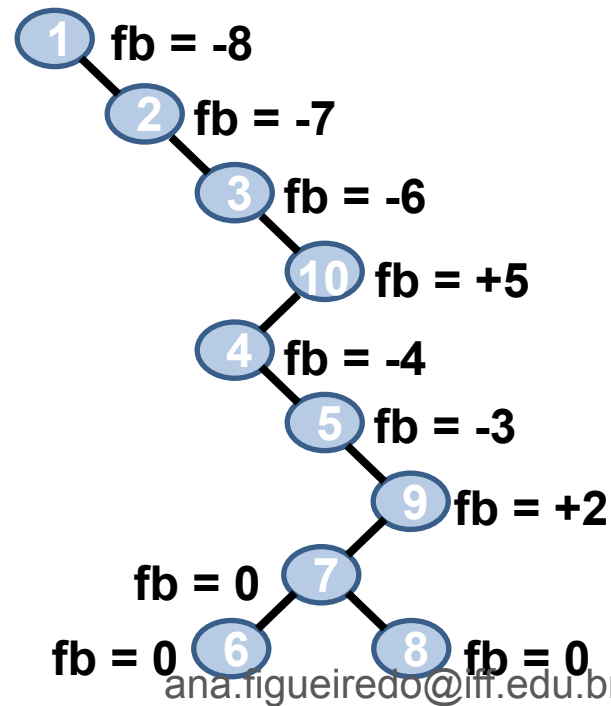
Achou o nó a ser removido. Tratar o tipo de remoção

Continua andando na árvore a procura do nó a ser removido

```
int remove_ArvBin(ArvBin *raiz, int valor){
    if(raiz == NULL) return 0;
    struct NO* ant = NULL;
    struct NO* atual = *raiz;
    while(atual != NULL){
        if(valor == atual->info){
            if(atual == *raiz)
                *raiz = remove_atual(atual);
            else{
                if(ant->dir == atual)
                    ant->dir = remove_atual(atual);
                else
                    ant->esq = remove_atual(atual);
            }
            return 1;
        }
        ant = atual;
        if(valor > atual->info)
            atual = atual->dir;
        else
            atual = atual->esq;
    }
    return 0;
}
```

Problema do balanceamento

- Infelizmente, os algoritmos de inserção e remoção em árvores binárias não garantem que a árvore gerada a cada passo esteja balanceada.
- Dependendo da ordem em que os dados são inseridos na árvore, podemos criar uma árvore na forma de uma escada
- Inserção dos valores {1,2,3,10,4,5,9,7,8,6}



Problema do balanceamento

- Solução para o problema de balanceamento
 - Modificar as operações de inserção e remoção de modo a balancear a árvore a cada nova inserção ou remoção.
 - Garantir que a diferença de alturas das sub-árvores esquerda e direita de cada nó seja de no máximo uma unidade
 - Exemplos de árvores balanceadas
 - Árvore AVL
 - Árvore 2-3-4
 - Árvore Rubro-Negra

- Definição

- Tipo de árvore binária balanceada com relação a altura das suas sub-árvores
- Criada por **Adelson-Velskii** e **Landis**, de onde recebeu a sua nomenclatura, em 1962
- Permite o rebalanceamento local da árvore
 - Apenas a parte afetada pela inserção ou remoção é rebalanceada
- Usa **rotações simples** ou **duplas** na etapa de rebalanceamento
 - Executadas a cada inserção ou remoção
 - As rotações buscam manter a árvore binária como uma árvore quase completa

59

-

$$FB = AE - AD$$

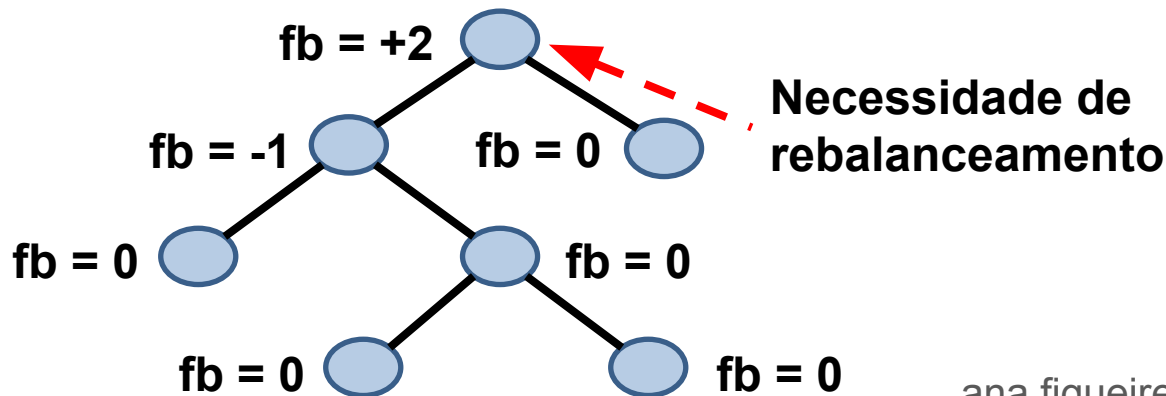
[



AD

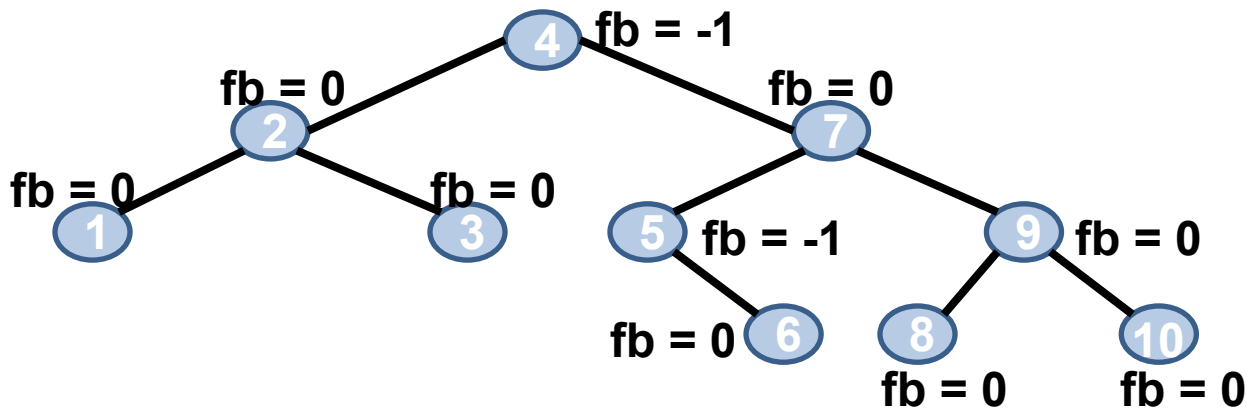
Árvore AVL

- As alturas das sub-árvores de cada nó diferem de no máximo uma unidade
 - O fator de balanceamento deve ser +1, 0 ou -1
 - Se **fb** > +1 ou **fb** < -1: a árvore deve ser balanceada naquele nó



Árvore AVL

- Voltando ao problema anterior
- Inserção dos valores {1,2,3,10,4,5,9,7,8,6}



TAD Árvore AVL

- Definindo a árvore
 - Criação e destruição: igual a da árvore binária

```
1 //Arquivo ArvoreAVL.h
2 typedef struct NO* ArvAVL;
3
4 //Arquivo ArvoreAVL.c
5 #include <stdio.h>
6 #include <stdlib.h>
7 #include "ArvoreAVL.h" //inclui os Protótipos
8 struct NO{
9     int info;
10    int alt; //altura daquela sub-árvore
11    struct NO *esq;
12    struct NO *dir;
13 };
14 //programa principal
15 ArvAVL* raiz; //ponteiro para ponteiro
```

TAD Árvore AVL

- Calculando o fator de balanceamento

```
1 //Funções auxiliares
2 //Calcula a altura de um nó
3 int alt_NO(struct NO* no) {
4     if(no == NULL)
5         return -1;
6     else
7         return no->alt;
8 }
9 //Calcula o fator de balanceamento de um nó
10 int fatorBalanceamento_NO(struct NO* no) {
11     return labs(alt_NO(no->esq) - alt_NO(no->dir));
12 }
```

Rotações

- Objetivo: corrigir o **fator de balanceamento** (ou **fb**) de cada nó
 - Operação básica para balancear uma árvore AVL
- Ao todo, existem dois tipos de rotação
 - Rotação simples
 - Rotação dupla

Rotações

- As rotações diferem entre si pelo sentido da inclinação entre o nó pai e filho
 - Rotação simples
 - O nó desbalanceado (pai), seu filho e o seu neto estão todos no mesmo sentido de inclinação
 - Rotação dupla
 - O nó desbalanceado (pai) e seu filho estão inclinados no sentido inverso ao neto
 - **Equivale a duas rotações simples.**

Rotações

- Ao todo, existem duas rotações simples e duas duplas:
 - Rotação **simples a direita** ou **Rotação LL**
 - Rotação **simples a esquerda** ou **Rotação RR**
 - Rotação **dupla a direita** ou **Rotação LR**
 - Rotação **dupla a esquerda** ou **Rotação RL**

Rotações

- Rotações são aplicadas no ancestral mais próximo do nó inserido cujo fator de balanceamento passa a ser +2 ou -2
 - Após uma inserção ou remoção, devemos voltar pelo mesmo caminho da árvore e recalcular o fator de balanceamento, **fb**, de cada nó
 - Se o **fb** desse nó for +2 ou -2, uma rotação deverá ser aplicada

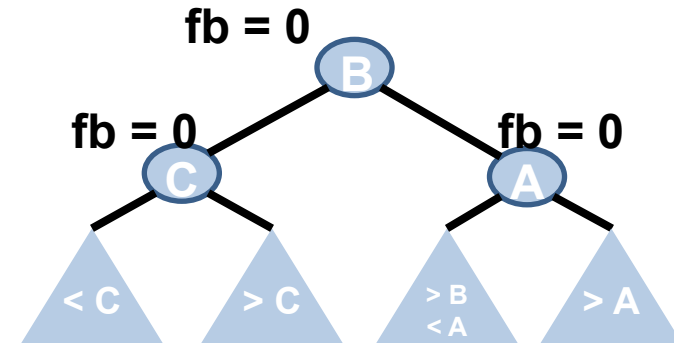
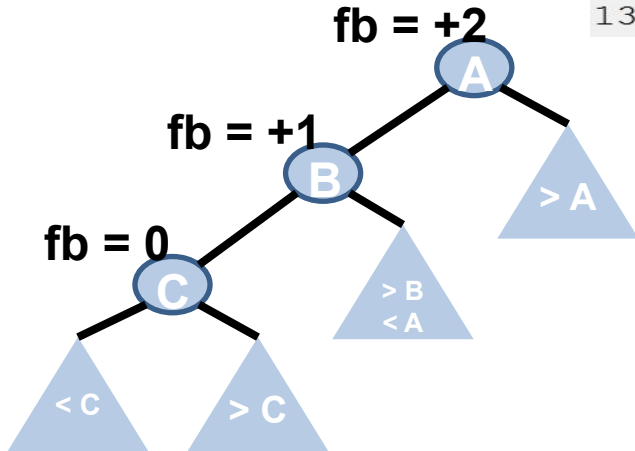
Rotação LL

- Rotação LL ou rotação simples à direita
 - Um novo nó é inserido na **sub-árvore da esquerda do filho esquerdo** de **A**
 - **A** é o nó desbalanceado
 - Dois movimentos para a esquerda: **LEFT LEFT**
 - É necessário fazer uma rotação à direita, de modo que o nó intermediário **B** ocupe o lugar de **A**, e **A** se torne a sub-árvore direita de **B**

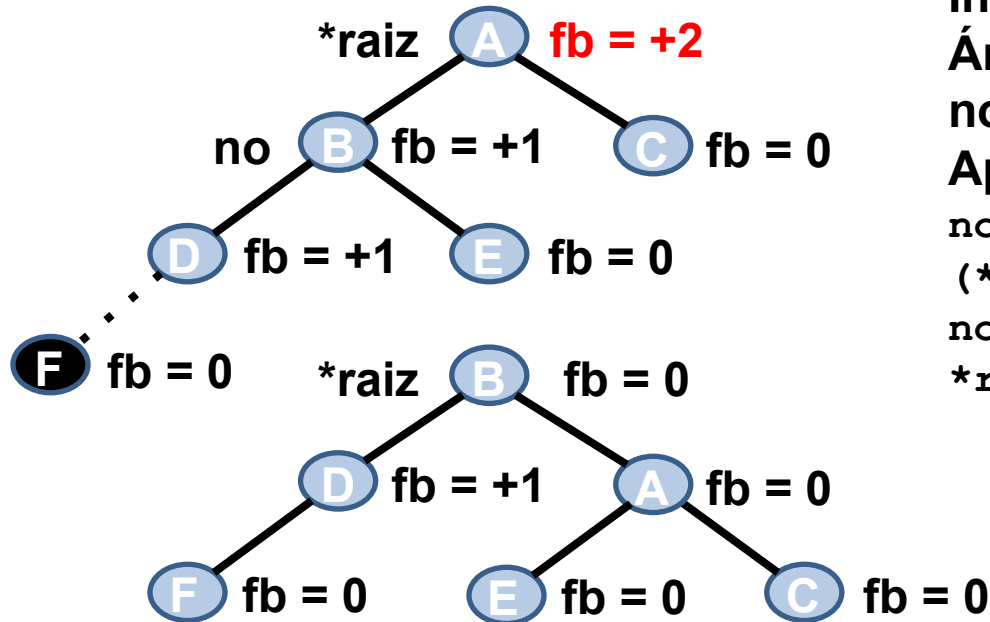
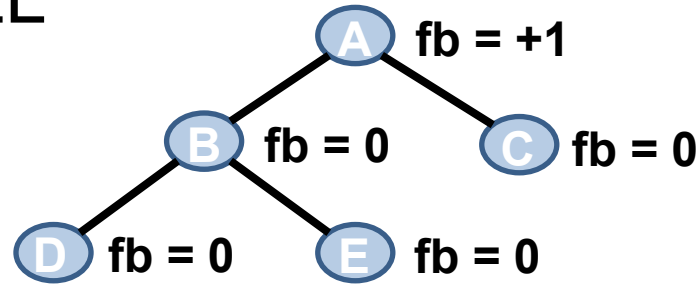
Rotação LL

- Exemplo

```
1 void RotacaoLL(ArvAVL *raiz) {  
2     struct NO *no;  
3     no = (*raiz)->esq;  
4     (*raiz)->esq = no->dir;  
5     no->dir = *raiz;  
6     (*raiz)->altura = maior(altura_NO((*raiz)->esq),  
7                             altura_NO((*raiz)->dir))  
8                             + 1;  
9     no->altura = maior(altura_NO(no->esq),  
10                      (*raiz)->altura) + 1;  
11     *raiz = no;  
12 }  
13
```



Rotação LL



Árvore AVL e fator de balanceamento de cada nó

Inserção do nó F na árvore
Árvore fica desbalanceada no nó A.

Aplicar Rotação LL no nó A

```
no = (*raiz)->esq;  
(*raiz)->esq = no->dir;  
no->dir = *raiz;  
*raiz = no;
```

Árvore Balanceada

ana.figueiredo@iff.edu.br

Rotação RR

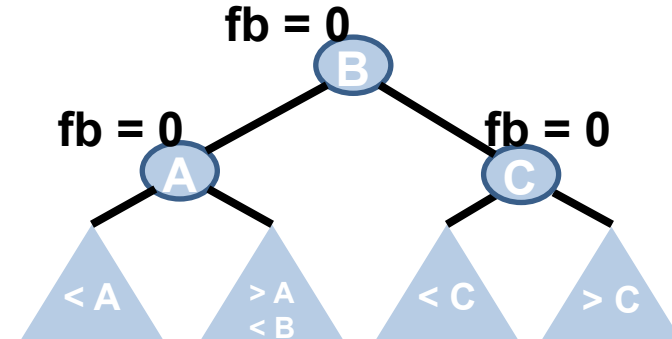
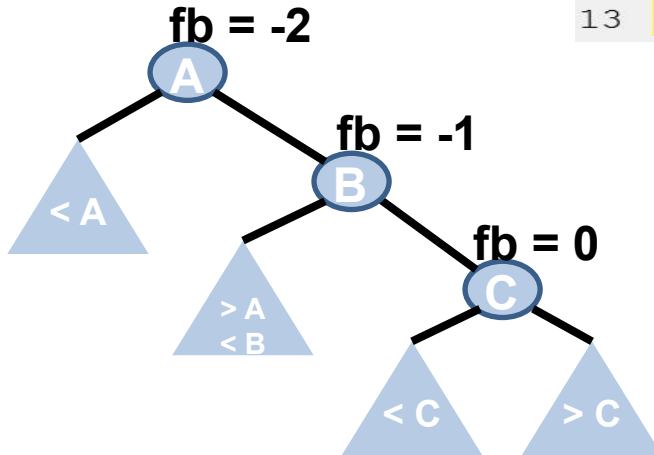
- Rotação RR ou rotação simples à esquerda
 - Um novo nó é inserido na **sub-árvore da direita do filho direito** de **A**
 - **A** é o nó desbalanceado
 - Dois movimentos para a direita: **RIGHT RIGHT**
 - É necessário fazer uma rotação à esquerda, de modo que o nó intermediário **B** ocupe o lugar de **A**, e **A** se torne a sub-árvore esquerda de **B**

Rotação RR

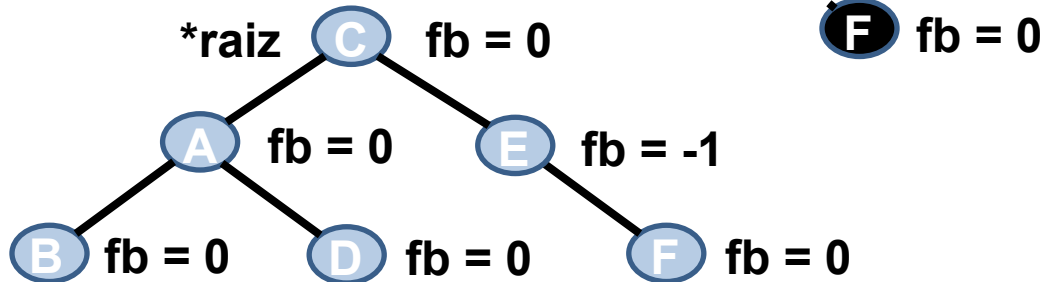
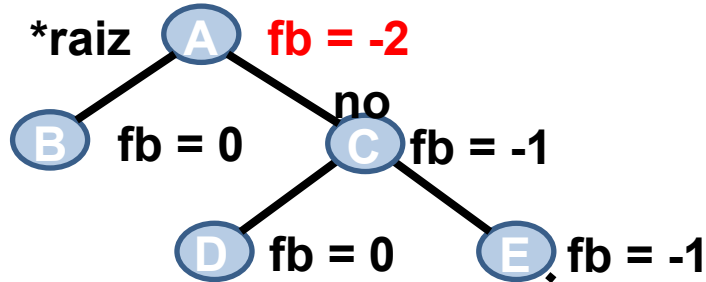
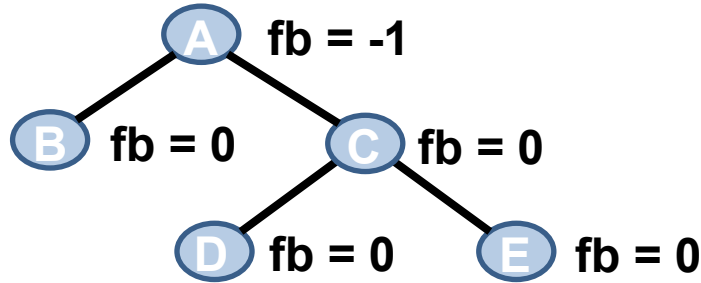
- Exemplo

```
1  
2  
3  
4  
5  
6  
7  
8  
9  
10  
11  
12  
13
```

```
void RotacaoRR(ArvAVL *raiz) {  
    struct NO *no;  
    no = (*raiz)->dir;  
    (*raiz)->dir = no->esq;  
    no->esq = (*raiz);  
    (*raiz)->altura = maior(altura_NO((*raiz)->esq),  
                             altura_NO((*raiz)->dir))  
        + 1;  
    no->altura = maior(altura_NO(no->dir),  
                       (*raiz)->altura) + 1;  
    (*raiz) = no;  
}
```



Rotação RR



Árvore balanceamento de cada nó

Inserção do nó F na árvore
Árvore fica desbalanceada no nó A.

Aplicar Rotação RR no nó A

```

no = (*raiz)->dir;
(*raiz)->dir = no->esq;
no->esq = (*raiz);
(*raiz) = no;
  
```

Árvore Balanceada

Rotação LR

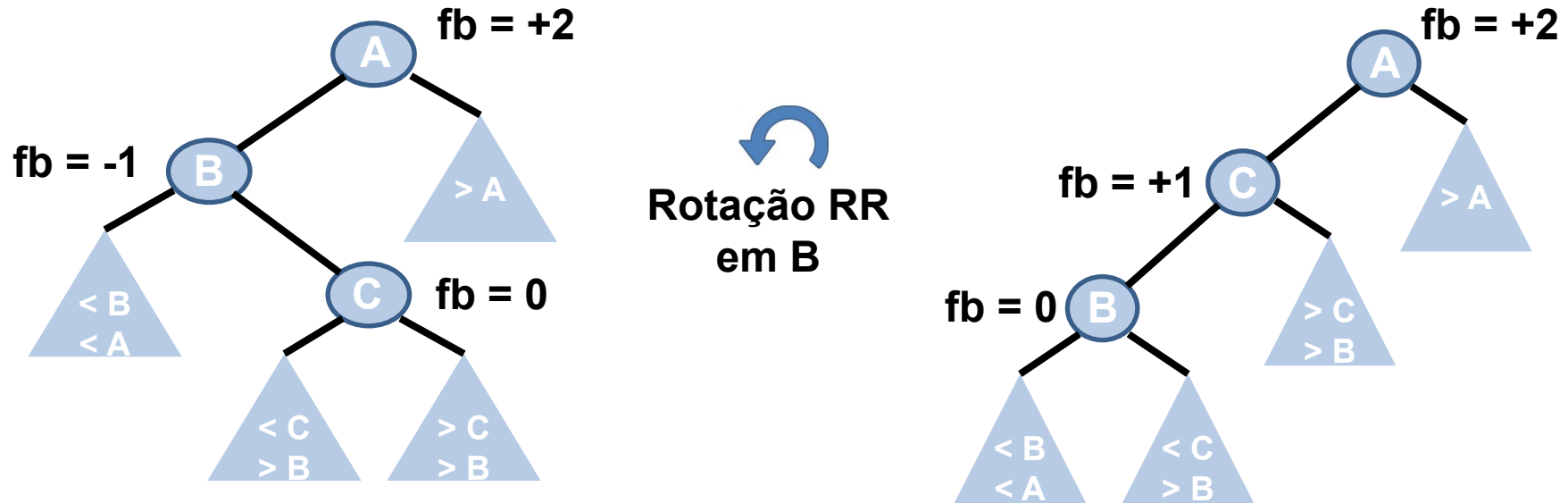
- Rotação LR ou rotação dupla à direita
 - Um novo nó é inserido na **sub-árvore da direita do filho esquerdo** de **A**
 - **A** é o nó desbalanceado
 - Um movimento para a esquerda e outro para a direita: **LEFT RIGHT**
 - É necessário fazer uma rotação dupla, de modo que o nó **C** se torne o pai dos nós **A** (filho da direita) e **B** (filho da esquerda)
 - Rotação RR em **B**
 - Rotação LL em **A**

Rotação LR

```
1  
2  
3  
4  
5
```

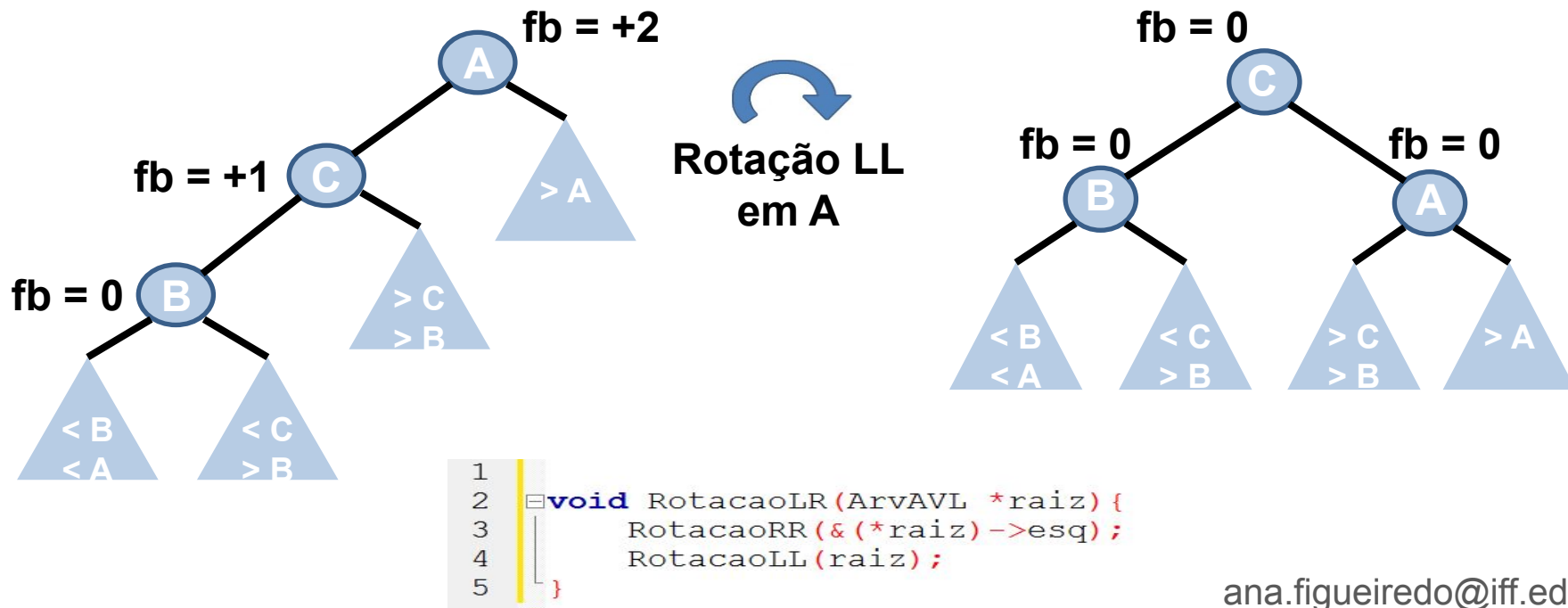
```
void RotacaoLR(ArvAVL *raiz) {  
    RotacaoRR(&(*raiz)->esq);  
    RotacaoLL(raiz);  
}
```

- Exemplo: primeira rotação



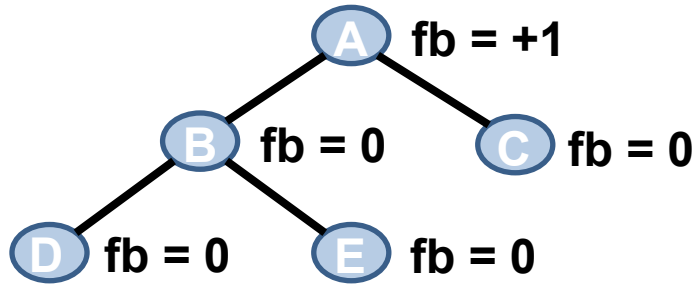
Rotação LR

- Exemplo: segunda rotação



Rotação LR

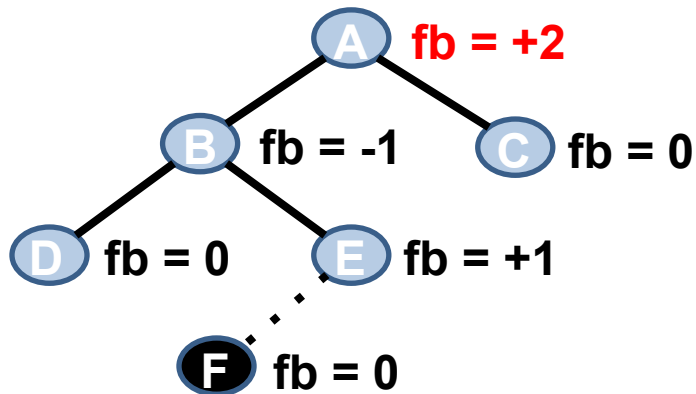
- Passo a passo



**Árvore AVL e fator de
balanceamento de cada nó**

Rotação LR

- Passo a passo



Inserção do nó F na árvore

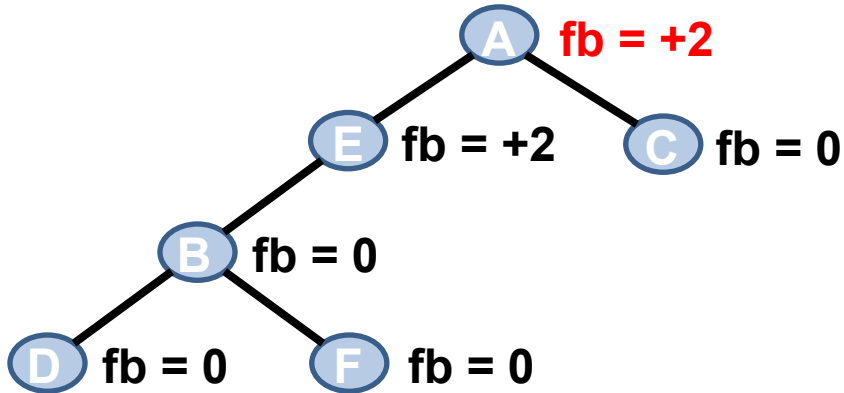
Árvore fica desbalanceada no nó A.

Aplicar Rotação LR no nó A.
Isso equivale a:

- Aplicar a Rotação RR no nó B
- Aplicar a Rotação LL no nó A

Rotação LR

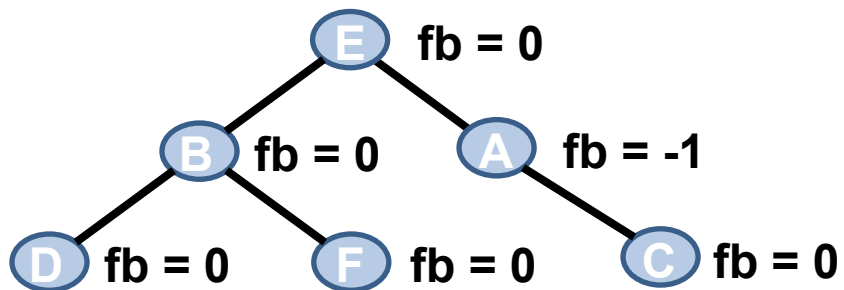
- Passo a passo



Árvore após aplicar a
Rotação RR no nó B

Rotação LR

- Passo a passo



**Árvore após aplicar a
Rotação LL no nó A**

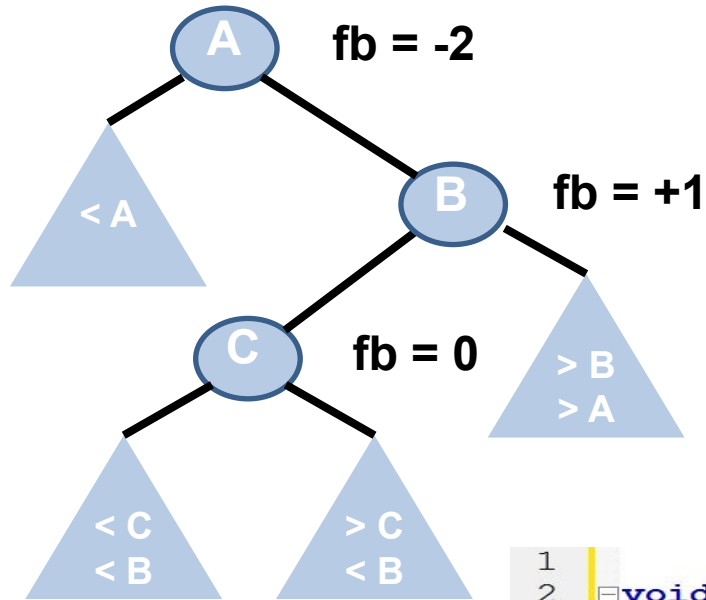
Árvore Balanceada

Rotação RL

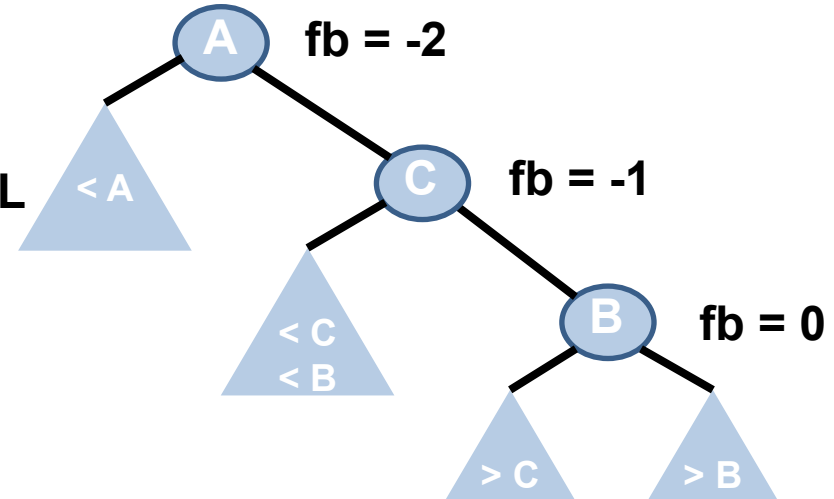
- Rotação RL ou rotação dupla à esquerda
 - um novo nó é inserido na **sub-árvore da esquerda do filho direito** de **A**
 - **A** é o nó desbalanceado
 - Um movimento para a direita e outro para a esquerda: **RIGHT LEFT**
 - É necessário fazer uma rotação dupla, de modo que o nó **C** se torne o pai dos nós **A** (filho da esquerda) e **B** (filho da direita)
 - Rotação LL em **B**
 - Rotação RR em **A**

Rotação RL

- Exemplo



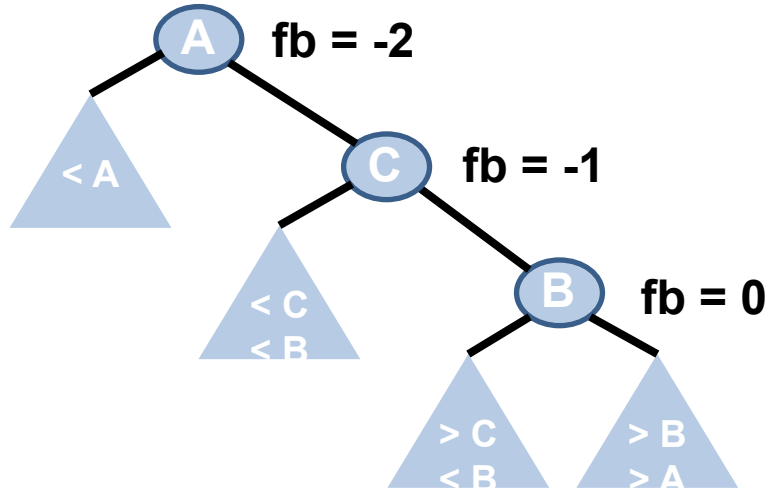

Rotação LL
em B



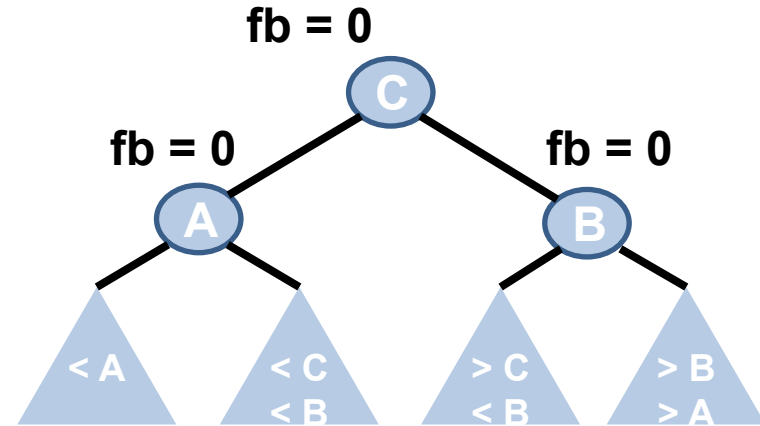
```
1  
2 void RotacaoRL (ArvAVL *raiz) {  
3     RotacaoLL (&(*raiz) -> dir);  
4     RotacaoRR (raiz);  
5 }
```

Rotação RL

- Exemplo




**Rotação RR
em A**



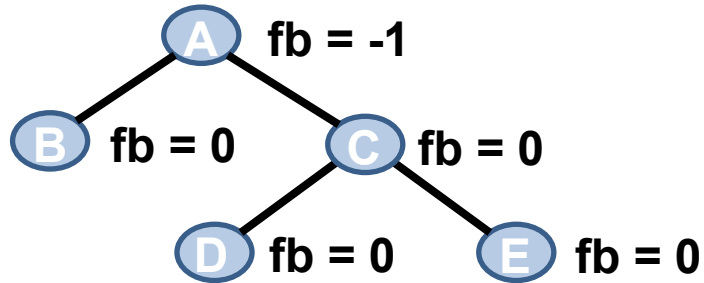
```

1
2 void RotacaoRL(ArvAVL *raiz) {
3     RotacaoLL(&(*raiz)->dir);
4     RotacaoRR(raiz);
5 }

```

Rotação RL

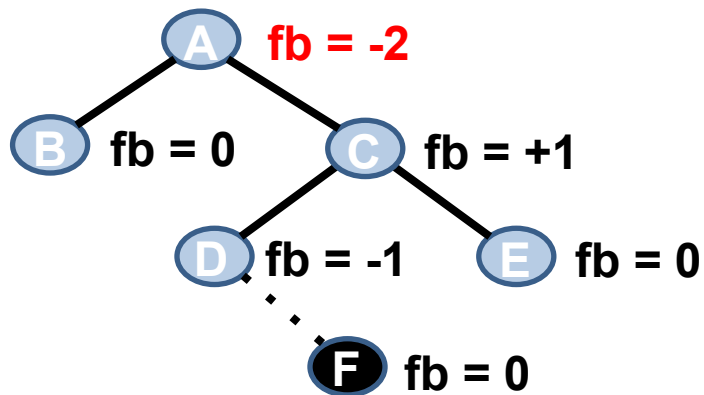
- Passo a passo



**Árvore AVL e fator de
balanceamento de cada nó**

Rotação RL

- Passo a passo



Inserção do nó F na árvore

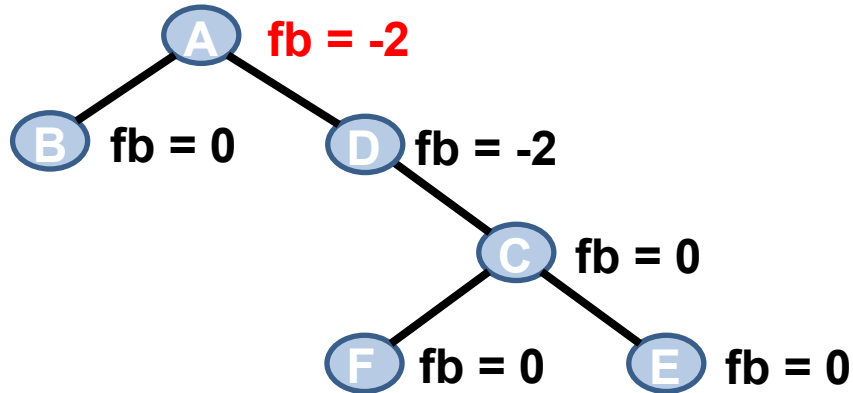
Árvore fica desbalanceada no nó A.

**Aplicar Rotação RL no nó A.
Isso equivale a:**

- **Aplicar a Rotação LL no nó C**
- **Aplicar a Rotação RR no nó A**

Rotação RL

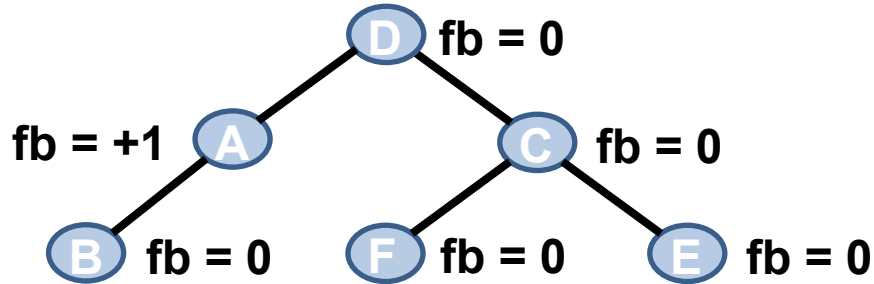
- Passo a passo



Árvore após aplicar a
Rotação LL no nó C

Rotação RL

- Passo a passo



**Árvore após aplicar a
Rotação RR no nó A**

Árvore Balanceada

Quando usar cada rotação?

- Uma dúvida muito comum é quando utilizar cada uma das quatro rotações

Fator de Balanceamento de A	Fator de Balanceamento de B	Posições dos nós B e C em relação ao nó A	Rotação
+2	+1	B é filho à esquerda de A C é filho à esquerda de B	LL
-2	-1	B é filho à direita de A C é filho à direita de B	RR
+2	-1	B é filho à esquerda de A C é filho à direita de B	LR
-2	+1	B é filho à de direita A C é filho à esquerda de B	RL

Quando usar cada rotação?

- Sinais iguais: rotação simples
 - Sinal positivo: rotação à direita (LL)
 - Sinal negativo: rotação à esquerda (RR)

Fator de Balanceamento de A	Fator de Balanceamento de B	Posições dos nós B e C em relação ao nó A	Rotação
+2	+1	B é filho à esquerda de A C é filho à esquerda de B	LL
-2	-1	B é filho à direita de A C é filho à direita de B	RR
+2	-1	B é filho à esquerda de A C é filho à direita de B	LR
-2	+1	B é filho à de direita A C é filho à esquerda de B	RL

Quando usar cada rotação?

- Sinais diferentes: rotação dupla
 - **A** positivo: rotação dupla a direita (LR)
 - **A** negativo: rotação dupla a esquerda (RL)

Fator de Balanceamento de A	Fator de Balanceamento de B	Posições dos nós B e C em relação ao nó A	Rotação
+2	+1	B é filho à esquerda de A C é filho à esquerda de B	LL
-2	-1	B é filho à direita de A C é filho à direita de B	RR
+2	-1	B é filho à esquerda de A C é filho à direita de B	LR
-2	+1	B é filho à de direita A C é filho à esquerda de B	RL

Árvore AVL: Inserção

- Para inserir um valor **V** na árvore
 - Se a raiz é igual a **NULL**, insira o nó
 - Se **V** é menor do que a raiz: vá para a **sub-árvore esquerda**
 - Se **V** é maior do que a raiz: vá para a **sub-árvore direita**
 - Aplique o método **recursivamente**
- Dessa forma, percorremos um conjunto de nós da árvore até chegar ao nó folha que irá se tornar o pai do novo nó
- Uma vez inserido o novo nó
 - Devemos voltar pelo caminho percorrido e calcular o fator de balanceamento de cada um dos nós visitados
 - Aplicar a rotação necessária para restabelecer o balanceamento da árvore se o fator de balanceamento for **+2** ou **-2**

Árvore AVL: Inserção

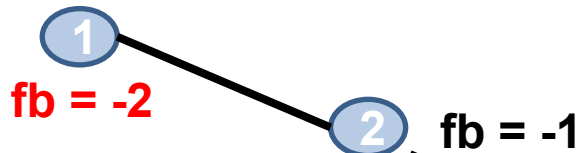
Inserir valor: 1



Inserir valor: 2

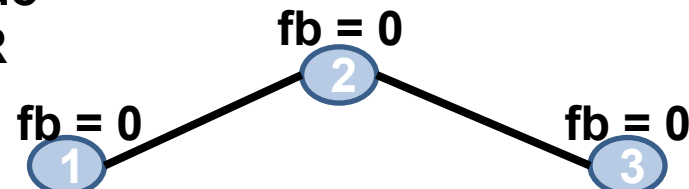


Inserir valor: 3



Nó "1" desbalanceado

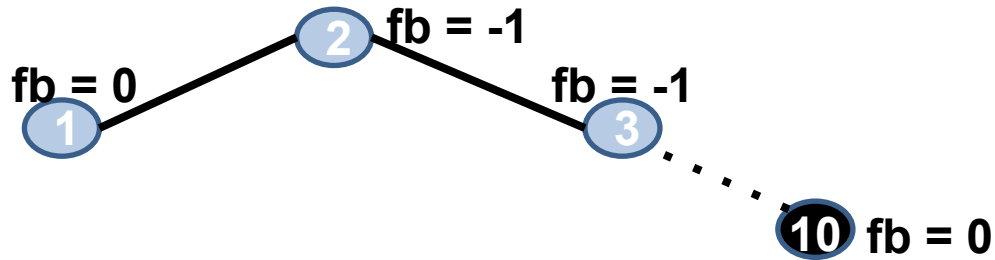
Aplicar Rotação RR



Árvore AVL: Inserção

- Passo a passo

Inserir valor: 10



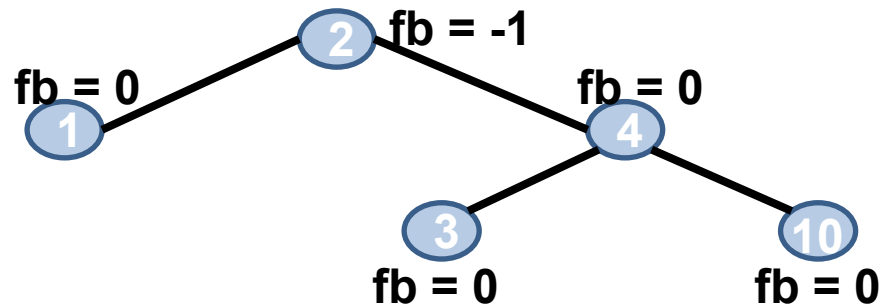
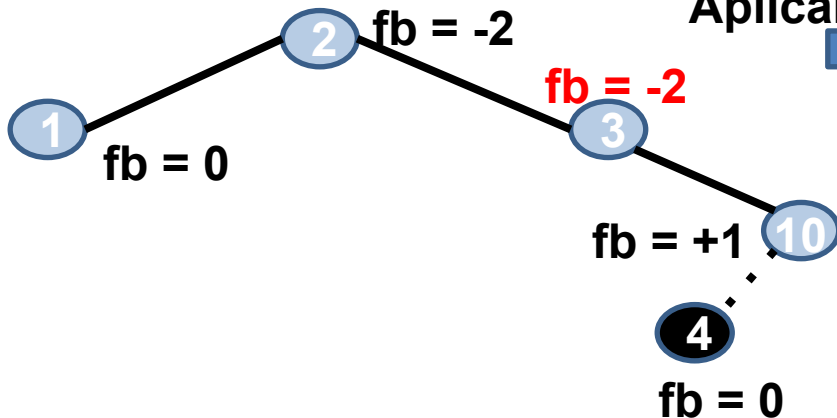
Árvore AVL: Inserção

94

- Passo a passo

Inserir valor: 4

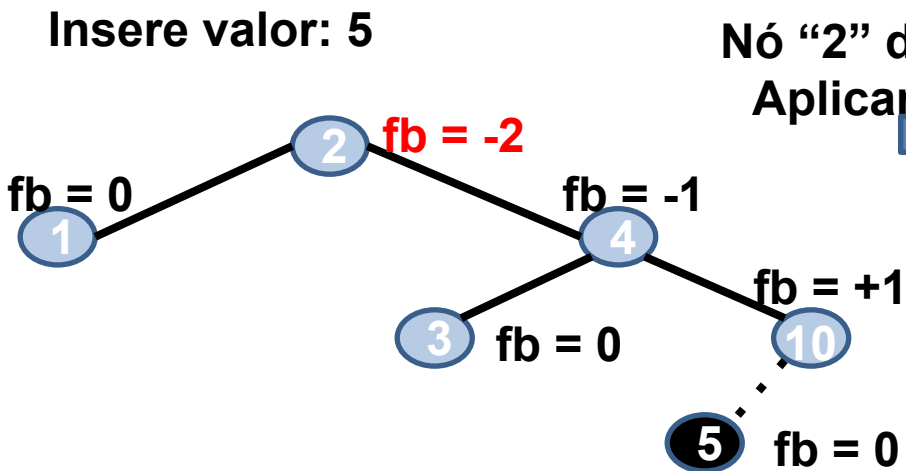
Nó "3" desbalanceado
Aplicar Rotação RL



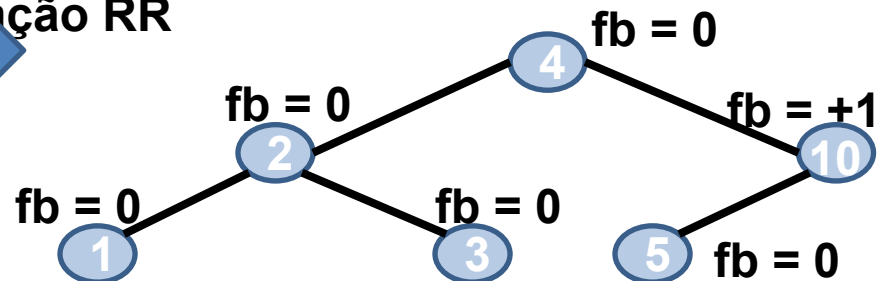
Árvore AVL: Inserção

- Passo a passo

Inserir valor: 5



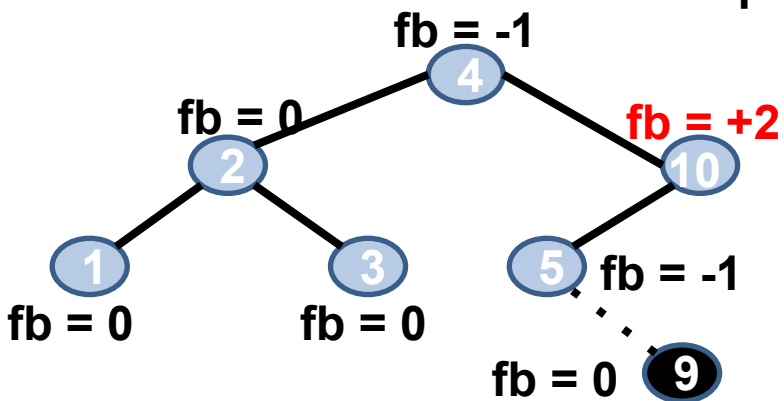
Nó "2" desbalanceado
Aplicar Rotação RR



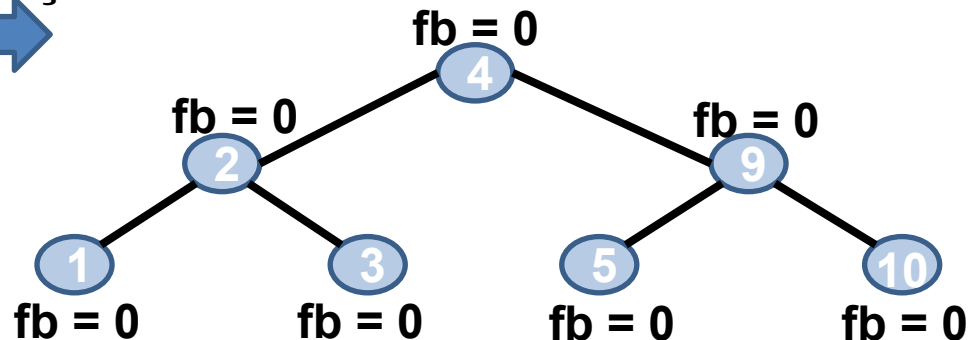
Árvore AVL: Inserção

- Passo a passo

Inserir valor: 9



Nó "10" desbalanceado
Aplicar Rotação LR



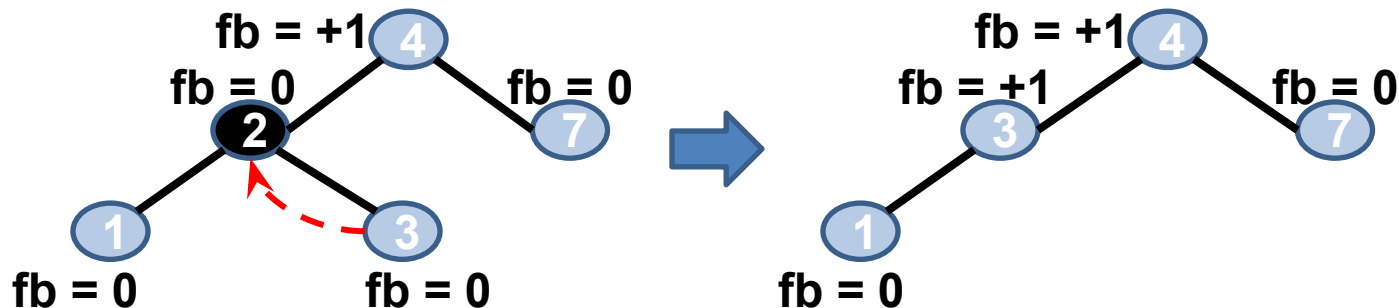
Árvore AVL: Remoção

- Como na inserção, temos que percorremos um conjunto de nós da árvore até chegar ao nó que será removido
 - Existem 3 tipos de remoção
 - Nó folha (sem filhos)
 - Nó com 1 filho
 - Nó com 2 filhos
- Uma vez removido o nó
 - Devemos voltar pelo caminho percorrido e calcular o fator de balanceamento de cada um dos nós visitados
 - Aplicar a rotação necessária para restabelecer o balanceamento da árvore se o fator de balanceamento for **+2** ou **-2**
 - **Remover** um nó da sub-árvore **direita** equivale a **inserir** um nó na sub-árvore **esquerda**
ana.figueiredo@iff.edu.br

Árvore AVL: Remoção

- Passo a passo

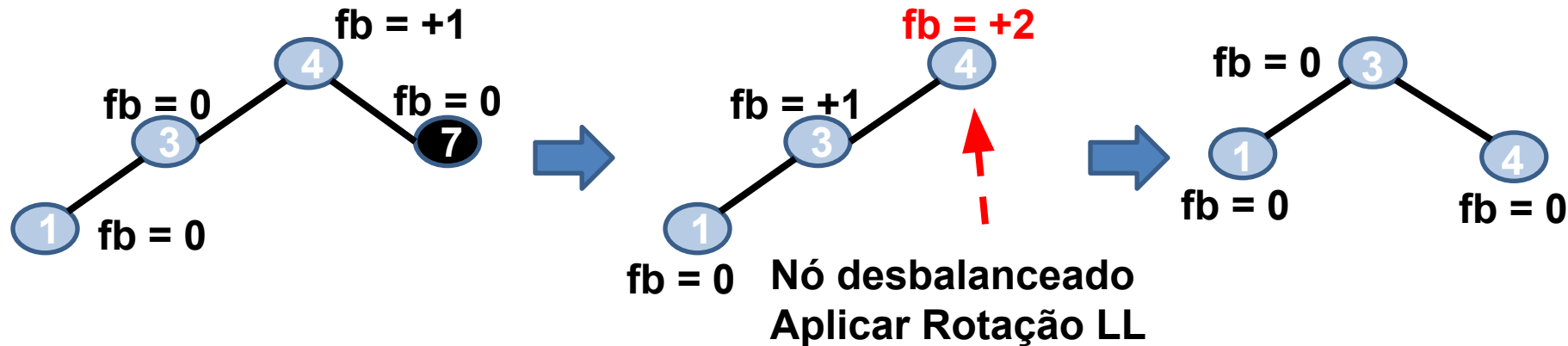
Remove valor: 2



Árvore AVL: Remoção

- Passo a passo

Remove valor: 7



Implementações

- Inserção

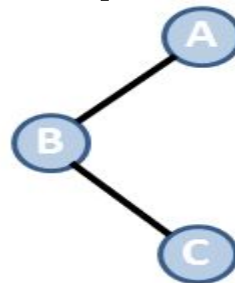
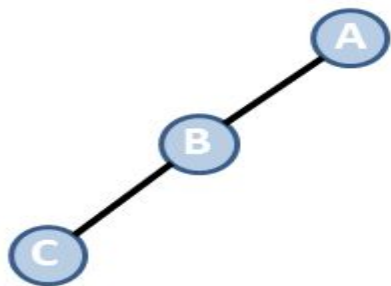
```
int insere_ArvAVL(ArvAVL *raiz, int valor){
    int res;
    if(*raiz == NULL){ //árvore vazia ou nó folha
        struct NO *novo;
        novo = (struct NO*) malloc(sizeof(struct NO));
        if(novo == NULL)
            return 0;

        novo->info = valor;
        novo->altura = 0;
        novo->esq = NULL;
        novo->dir = NULL;
        *raiz = novo;
        return 1;
    }
    //continua...
```

TAD Árvore AVL

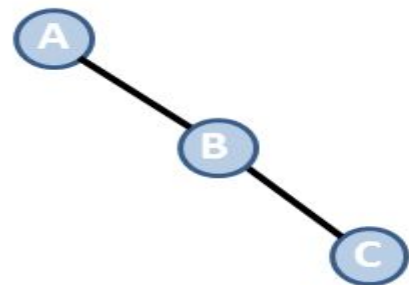
- Inserção

```
//continuação
struct NO *atual = *raiz;
if(valor < atual->info){
    if((res=inserere_ArvAVL(&(atual->esq), valor))==1){
        if(fatorBalanceamento_NO(atual) >= 2){
            if(valor < (*raiz)->esq->info ){
                RotacaoLL(raiz);
            } else{
                RotacaoLR(raiz);
            }
        }
    }
}
```

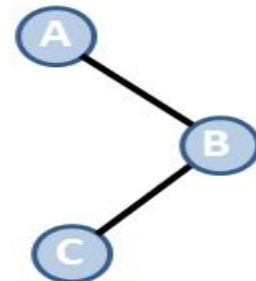


TAD Árvore AVL

- Inserção



```
//continuação
else{
    if(valor > atual->info){
        if((res=inserere_ArvAVL(&(atual->dir), valor))==1){
            if(fatorBalanceamento_NO(atual) >= 2){
                if((*raiz)->dir->info < valor){
                    RotacaoRR(raiz);
                }else{
                    RotacaoRL(raiz);
                }
            }
        }else{
            printf("Valor duplicado!!\n");
            return 0;
        }
    }
    atual->altura = maior(altura_NO(atual->esq),
                        altura_NO(atual->dir)) + 1;
    return res;
}
```



Árvore AVL: Inserção

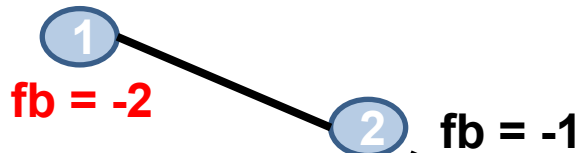
Inserir valor: 1



Inserir valor: 2

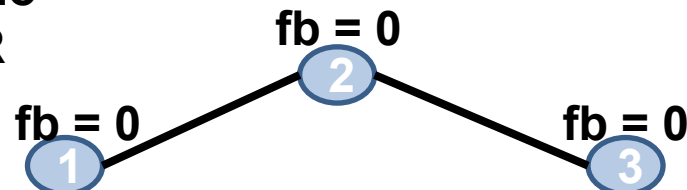


Inserir valor: 3



Nó "1" desbalanceado

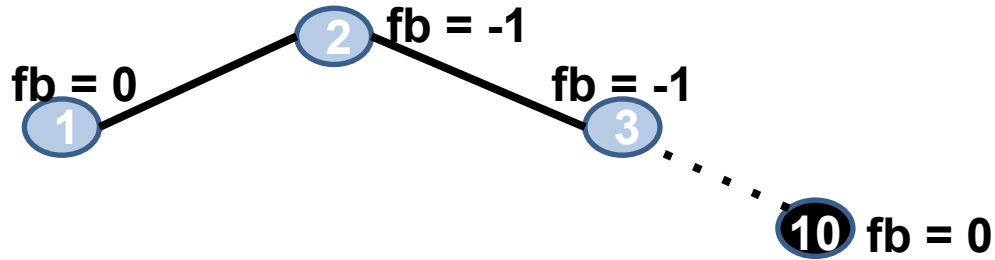
Aplicar Rotação RR



Árvore AVL: Inserção

- Passo a passo

Inserir valor: 10



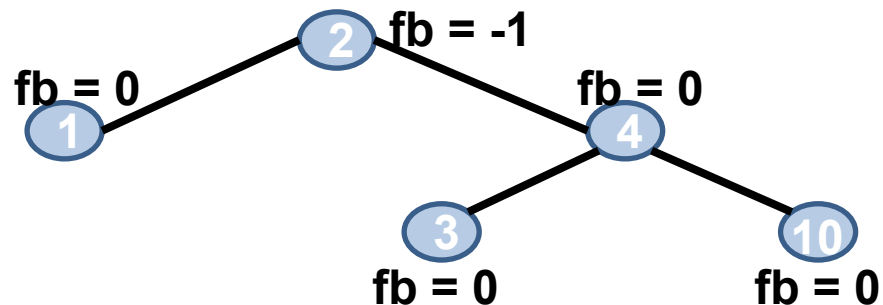
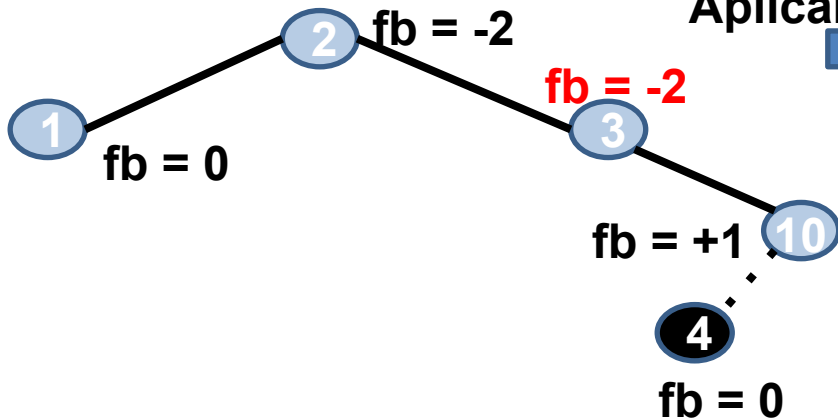
Árvore AVL: Inserção

106

- Passo a passo

Inserir valor: 4

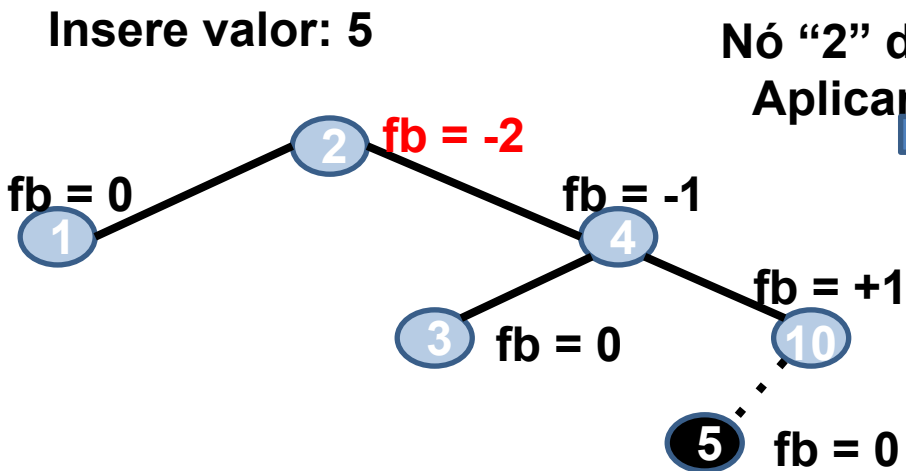
Nó "3" desbalanceado
Aplicar Rotação RL



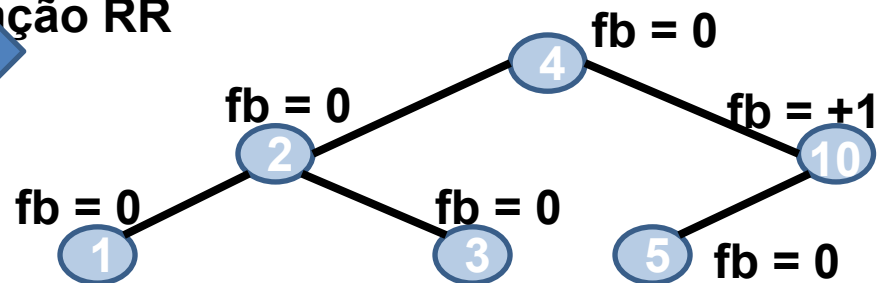
Árvore AVL: Inserção

- Passo a passo

Inserir valor: 5



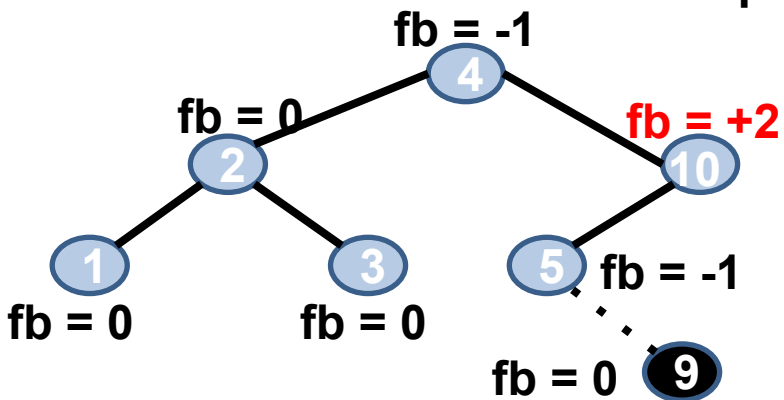
Nó "2" desbalanceado
Aplicar Rotação RR



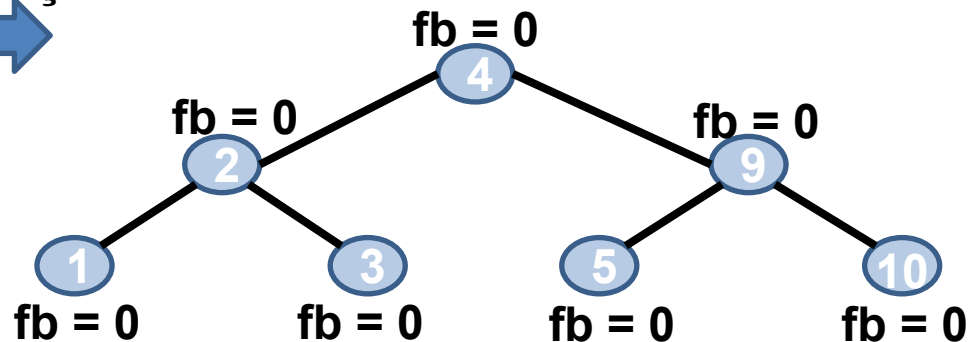
Árvore AVL: Inserção

- Passo a passo

Inserir valor: 9



Nó "10" desbalanceado
Aplicar Rotação LR



Árvore AVL: Remoção

- Como na inserção, temos que percorremos um conjunto de nós da árvore até chegar ao nó que será removido
 - Existem 3 tipos de remoção
 - Nó folha (sem filhos)
 - Nó com 1 filho
 - Nó com 2 filhos
- Uma vez removido o nó
 - Devemos voltar pelo caminho percorrido e calcular o fator de balanceamento de cada um dos nós visitados
 - Aplicar a rotação necessária para restabelecer o balanceamento da árvore se o fator de balanceamento for **+2** ou **-2**
 - **Remover** um nó da sub-árvore **direita** equivale a **inserir** um nó na sub-árvore **esquerda**

TAD Árvore AVL

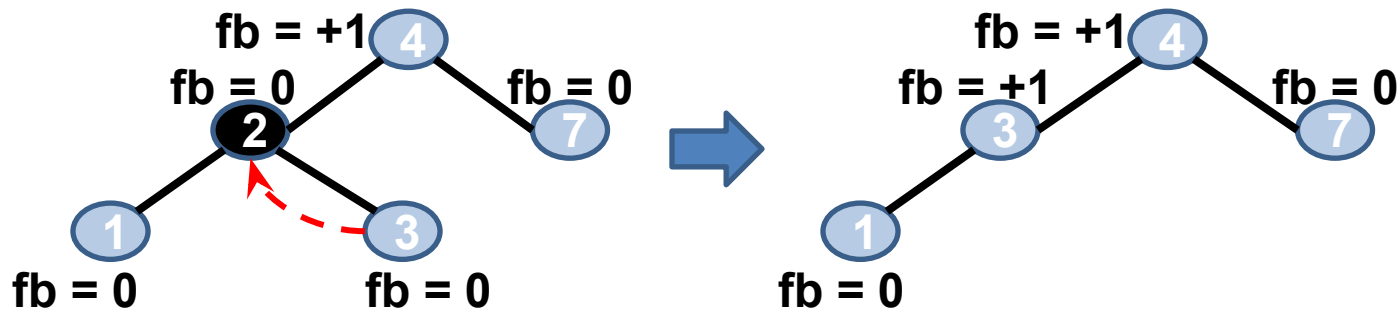
- Remoção
 - Trabalha com 2 funções
 - Busca pelo nó
 - Remoção do nó com 2 filhos

```
8  int remove_ArvAVL(ArvAVL *raiz, int valor) {
9      /*
10     FUNÇÃO RESPONSÁVEL PELA BUSCA
11     DO NÓ A SER REMOVIDO
12     */
13 }
14 struct NO* procuraMenor(struct NO* atual) {
15     /*
16     FUNÇÃO RESPONSÁVEL POR TRATAR OS
17     A REMOÇÃO DE UM NÓ COM 2 FILHOS
18     */
19 }
```

Árvore AVL: Remoção

- Passo a passo

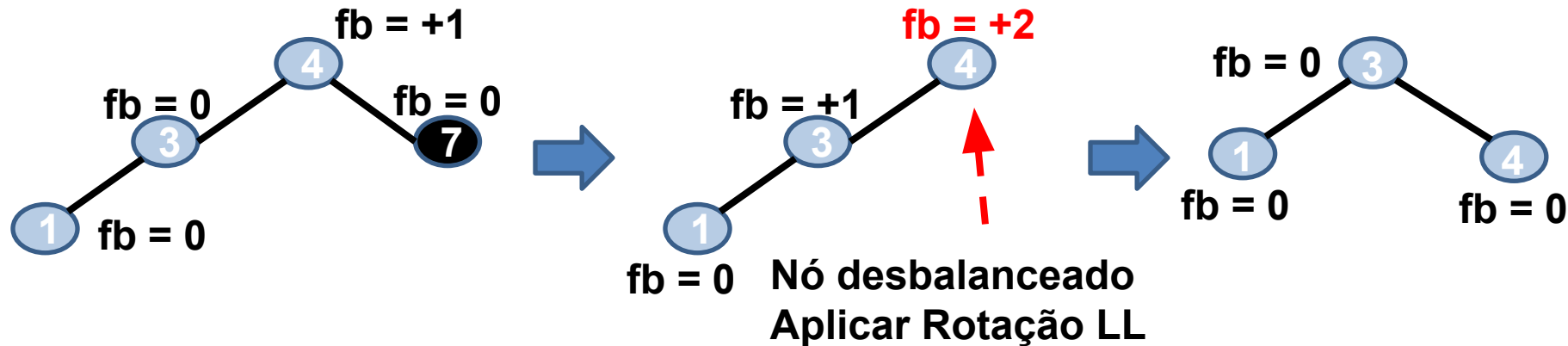
Remove valor: 2



Árvore AVL: Remoção

- Passo a passo

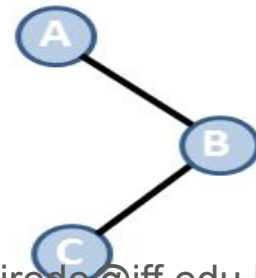
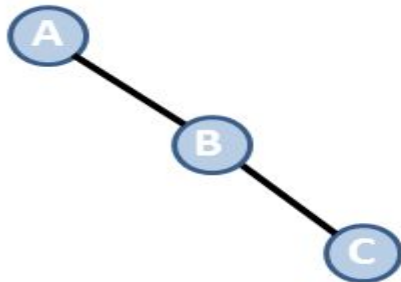
Remove valor: 7



TAD Árvore AVL

- Remoção

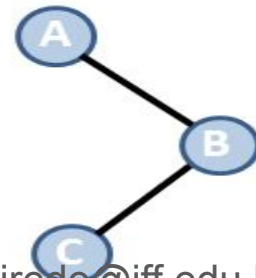
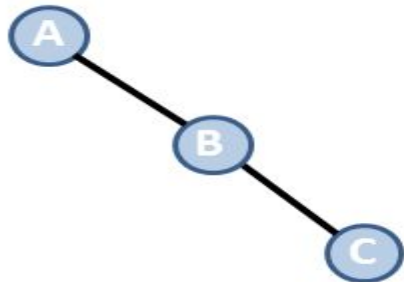
```
int remove_ArvAVL(ArvAVL *raiz, int valor) {  
    if(*raiz == NULL) { // valor não existe  
        printf("valor não existe!!\n");  
        return 0;  
    }  
    int res;  
    if(valor < (*raiz)->info) {  
        if((res=remove_ArvAVL(&(*raiz)->esq, valor))==1)  
            if(fatorBalanceamento_NO(*raiz) >= 2) {  
                if(altura_NO((*raiz)->dir->esq)  
                   <= altura_NO((*raiz)->dir->dir))  
                    RotacaoRR(raiz);  
                else  
                    RotacaoRL(raiz);  
            }  
    }  
    //continua...  
}
```



TAD Árvore AVL

- Remoção

```
int remove_ArvAVL(ArvAVL *raiz, int valor) {  
    if(*raiz == NULL) { // valor não existe  
        printf("valor não existe!!\n");  
        return 0;  
    }  
    int res;  
    if(valor < (*raiz)->info) {  
        if((res=remove_ArvAVL(&(*raiz)->esq, valor))==1)  
            if(fatorBalanceamento_NO(*raiz) >= 2) {  
                if(altura_NO((*raiz)->dir->esq)  
                   <= altura_NO((*raiz)->dir->dir))  
                    RotacaoRR(raiz);  
                else  
                    RotacaoRL(raiz);  
            }  
    }  
    //continua...  
}
```



TAD Árvore AVL

- Remoção

```
//continuação...  
if((*raiz)->info < valor){  
    if((res=remove_ArvAVL(&(*raiz)->dir, valor))==1){  
        if(fatorBalanceamento_NO(*raiz) >= 2){  
            if(altura_NO((*raiz)->esq->dir)  
                <= altura_NO((*raiz)->esq->esq) )  
                RotacaoLL(raiz);  
            else  
                RotacaoLR(raiz);  
        }  
    }  
}  
//continua...
```



TAD Árvore AVL

● Remoção

Pai tem 1 ou
nenhum filho

Pai tem 2 filhos:
Substituir pelo nó
mais a esquerda
da sub-árvore da
direita

Corrige a
altura

```
if ((*raiz)->info == valor) {
    if ((*raiz)->esq == NULL || (*raiz)->dir == NULL) { // nó tem 1 filho ou nenhum
        struct NO *oldNode = (*raiz);
        if ((*raiz)->esq != NULL)
            *raiz = (*raiz)->esq;
        else
            *raiz = (*raiz)->dir;
        free(oldNode);
    } else { // nó tem 2 filhos
        struct NO* temp = procuraMenor((*raiz)->dir);
        (*raiz)->info = temp->info;
        remove_ArvAVL(&(*raiz)->dir, (*raiz)->info);
        if (fatorBalanceamento_NO(*raiz) >= 2) {
            if (altura_NO((*raiz)->esq->dir) <= altura_NO((*raiz)->esq->esq))
                RotacaoLL(raiz);
            else
                RotacaoLR(raiz);
        }
    }
    if (*raiz != NULL)
        (*raiz)->altura = maior(altura_NO((*raiz)->esq),
                                altura_NO((*raiz)->dir)) + 1;

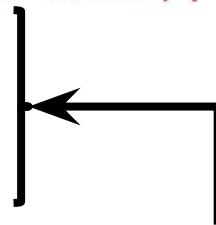
    return 1;
}
(*raiz)->altura = maior(altura_NO((*raiz)->esq),
                        altura_NO((*raiz)->dir)) + 1;

return res;
}
```

TAD Árvore AVL

- Remoção

```
struct NO* procuraMenor(struct NO* atual) {  
    struct NO *no1 = atual;  
    struct NO *no2 = atual->esq;  
    while (no2 != NULL) {  
        no1 = no2;  
        no2 = no2->esq;  
    }  
    return no1;  
}
```



Procura pelo nó
mais á esquerda